IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received 17 February 2025, accepted 14 March 2025, date of publication 24 March 2025, date of current version 2 April 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3553626

RESEARCH ARTICLE

Understanding Flaky Tests Through Linguistic Diversity: A Cross-Language and Comparative Machine Learning Study

AZEEM AHMAD[1], XIN SUN[2], MUHAMMAD RASHID NAEEM[1], YASIR JAVED[1], (Member, IEEE), MOHAMMAD AKOUR[1], AND KRISTIAN SANDAHL[2], (Senior Member, IEEE)

[1]Department of Software Engineering, Prince Sultan University, Riyadh 11586, Saudi Arabia
[2]Department of Computer Sciences, Linköping University, 58185 Linköping, Sweden

Corresponding author: Azeem Ahmad (aahmad@psu.edu.sa)

This work was supported by the Prince Sultan University.

ABSTRACT Software development is significantly impeded by flaky tests, which intermittently pass or fail without requiring code modifications, resulting in a decline in confidence in automated testing frameworks. Code smells (i.e., test case or production code) are the primary cause of test flakiness. In order to ascertain the prevalence of test smells, researchers and practitioners have examined numerous programming languages. However, one isolated experiment was conducted, which focused solely on one programming language. Across a variety of programming languages, such as Java, Python, C++, Go, and JavaScript, this study examines the predictive accuracy of a variety of machine learning classifiers in identifying flaky tests. We compare the performance of classifiers such as Random Forest, Decision Tree, Naive Bayes, Support Vector Machine, and Logistic Regression in both single-language and cross-language settings. In order to ascertain the impact of linguistic diversity on the flakiness of test cases, models were trained on a single language and subsequently tested on a variety of languages. The following key findings indicate that Random Forest and Logistic Regression consistently outperform other classifiers in terms of accuracy, adaptability, and generalizability, particularly in cross-language environments. Additionally, the investigation contrasts our findings with those of previous research, exhibiting enhanced precision and accuracy in the identification of flaky tests as a result of meticulous classifier selection. We conducted a thorough statistical analysis, which included t-tests, to assess the importance of classifier performance differences in terms of accuracy and F1-score across a variety of programming languages. This analysis emphasizes the substantial discrepancies between classifiers and their effectiveness in detecting flaky tests. The datasets and experiment code utilized in this study are accessible through an open source GitHub repository to facilitate reproducibility is available at: https://github.com/PELAB-LiU/FlakyCrossLanguage. Our results emphasize the effectiveness of probabilistic and ensemble classifiers in improving the reliability of automated testing, despite certain constraints, including the potential biases introduced by language-specific structures and dataset variability. This research provides developers and researchers with practical insights that can be applied to the mitigation of flaky tests in a variety of software environments.

INDEX TERMS Flaky tests, non-deterministic tests, machine learning, artificial intelligence, software testing.

I. INTRODUCTION

Developers must ensure that any changes they make to the code base do not introduce new software flaws or interfere with current functionality. A number of rigorous tests are

The associate editor coordinating the review of this manuscript and approving it for publication was Porfirio Tramontana.

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

required to confirm that their changes are correct. When a test case fails, developers assume that it was caused by recent code changes. However, it is not uncommon for certain test failures to be unrelated to code changes. These failures can be attributed to a phenomenon known as ''test flakiness'' (TF), which occurs when tests fail inconsistently despite the absence of actual defects in the code. Consequently, developers often have to allocate significant time and effort into analyzing their modifications, attempting to identify the underlying reason of the failure, reaching the conclusion that the failure is due to flakiness in the test suite [1], [2].

In literature, the most common definition of a flaky test is: *a test that exhibits both pass and fail outcomes when no changes are introduced into the code base*. According to King et al. [3], ''flaky tests exhibit both passing and failing results when neither the code nor the test has changed''.

Finding the root causes of test flakiness also results in frustration, in addition to limiting developer productivity, because significant development time is wasted on correcting false positives rather than addressing real problems in the code base [2]. As a result, reducing test flakiness is critical for increasing the reliability of automated tests and allowing developers to focus on critical debugging and improvement activities. Larger-scale software still shows significant test flakiness. For example, flaky tests at Google account for 4.56% of the total 1.6 million failed test cases [4]. According to [5] and [6], flaky test suites caused 13% and 50%, respectively, of all build failures. According to [7], one out of every seven test cases created by Google engineers fails due to flakiness. Developers have expressed growing mistrust in their test suite due to frequent flakiness [2]. Unpredictability of test results, such as intermittent failures without code modifications, undermines trust in automated testing [2]. Lack of trust can lead developers to ignore test failures or spend too much time troubleshooting issues that do not exist, reducing overall development efficiency [2]. Several examples of flaky and non-flaky tests have been provided by Ahmad et al. [2].

The research conducted in test flakiness (TF) is about empirical analysis (e.g., studies on open source software) of test smells [4], [8], tools or techniques to detect test smells related to test flakiness [9], [10], [11], [12], machine learning approaches to predicting TF [3], and to draw researchers' attention to TF [13]. The main root causes of test flakiness are due to test case code, product code, or within testing infrastructure, with the majority coming from test case code, particularly with test smells [2]. Test code smells are indicators that point to deeper issues in your code.

This study presents a novel approach to detecting flaky tests by looking at how test cases written in different programming languages (Java, Python, C++, Go, and JavaScript) affect test flakiness. While previous research has primarily focused on identifying test smells or code characteristics within a single language or on using limited feature sets as indicators of test flakiness, our study broadens the scope by investigating how linguistic diversity affects flaky test detection. We want to see if exposing classifiers to different codebases can improve the robustness and generalization of flaky test predictions.

Previous research has frequently focused on vocabulary-based analysis and testing smells within specific languages. For example, Pinto et al. examined flaky test vocabularies and showed that certain terms could predict flakiness in Java codebases [14]. Camara et al. replicated the experiments and confirmed the link between test vocabulary and flakiness, but did not investigate cross-language generalizability [15]. Our research deviates by examining whether training classifiers on multi-language datasets can capture shared flakiness patterns that transcend language-specific syntax, potentially enabling classifier performance gains across diverse codebases.

Furthermore, we aim to validate and expand on findings from these and other studies that have highlighted the vocabulary and structural attributes of flaky tests [14], [15]. By replicating and expanding these experiments with a diverse, multi-language dataset, we hope to determine whether vocabulary-based and structural indicators of test flakiness identified in previous work apply across languages or if their predictive power is reduced in a multi-language setting. Our findings may point to broader, language-independent indicators of flakiness, allowing developers to detect and mitigate flaky tests more effectively.

Apart from assessing machine learning classifiers for flaky test identification, we also included a thorough statistical analysis to understand the relevance of performance variations among several classifiers and programming languages. We investigated statistically significant trends and variations in classifier accuracy and F1-scores by means of paired t-tests. This method provides deeper understanding of the efficacy of classifiers in language-specific and cross-language situations, so strengthening the validity of our results by quantitatively proving which ones routinely outperform others. The outcome of this paper is as follows: This study attempts to answer the following questions:

- **RQ 1.1:** *How effective are machine learning classifiers in detecting flaky tests within individual programming languages?*
- **RQ 1.2:** *What is the impact of combining different programming languages on the predictive accuracy of flaky test classifiers?*
- **RQ 1.3:** *What are the common and unique features that contribute to flaky test detection across different programming languages, and how do these features impact classifier performance?*
- **RQ 1.4:** *How does varying training and testing percentages impact the predictive accuracy of machine learning classifiers for flaky test detection across different languages and combined datasets?*

The objective of this questioning is to assess the efficacy of classifiers in detecting flaky tests when they are trained and tested within the same language, as well as in circumstances

where datasets from multiple languages are utilized. Our objective is to determine the classifiers that exhibit the highest performance in cross-language and language-specific contexts.

- **RQ 2:** *To what extent does the predictive accuracy of machine learning classifiers vary when trained on test cases from one programming language and tested on those from another?*

This research question examines the feasibility of cross-language learning by training classifiers in one language (e.g., Java) and evaluating them in another (e.g., Python). It also aids in comprehending the generalizability of machine learning models in identifying flakiness across programming languages and in recognizing common, language-independent patterns of flakiness.

- **RQ 3:** *How does the predictive performance of our study's models compare with the findings of prior work [14], [15] on the vocabulary of flaky tests?*

The objective of this research question is to verify the validity of prior research that has established a correlation between flaky tests and specific vocabulary or code smells. This will be accomplished by implementing the results in a multilingual setting. This comparison will assist in determining whether the flakiness indicators identified in previous research are consistent across languages or if they vary significantly depending on the language used.

- **RQ 4:** *What is the statistical significance of performance differences among machine learning classifiers in detecting flaky tests, and how do these differences vary across programming languages?*

In this question, we incorporated a detailed statistical analysis to understand the significance of performance differences across various classifiers and programming languages.

The datasets and experiment code used in this study is provided in an open source GitHub repo[1] for reproducibility. The detailed contributions of this work are as follows:

- **C1:** The highlights about which classifiers are more reliable for language-specific flaky test detection and points to the nuances in behavior across programming ecosystems (RQ 1.1).
- **C2:** Combining datasets from several programming languages and their impact on the generalizability of certain classifiers while retaining consistent performance for others (RQ 1.2).
- **C3:** Understanding the interplay between common and unique features is key to improving both language-specific and general-purpose models (RQ 1.3).
- **C4:** The study provides insights into the trade-offs between training data size and classifier performance consistency (RQ 1.4).
- **C5:** The study identifies classifiers with higher potential for generalizability and the importance of shared patterns across languages (RQ 2).
- **C6:** This study achieved a comparative advantage over prior work in certain metrics, reflecting the benefits of updated methodologies and feature engineering (RQ 3).

[1] https://github.com/PELAB-LiU/FlakyCrossLanguage

II. RELATED WORK

A. UNDERSTANDING FLAKY TESTS AND THEIR CAUSES

Flaky tests have been recognized as a significant challenge in software testing. Ahmad et al. [2] conducted an empirical analysis of flaky test causes, identifying factors like non-deterministic behaviors and resource contention. Luo et al. [4] and Shi et al. [10] extended this understanding by analyzing flaky tests in large-scale projects, focusing on timing-related and environment-dependent flakiness.

Expanding on these studies, Pinto et al. [14] introduced a vocabulary of flaky tests, categorizing them into asynchronous and order-dependent types. Camara et al. [15] validated this vocabulary in diverse projects. This aligns with Marinov et al. [16], who emphasized flaky tests' impact on regression testing, and Hilton et al. [6], who analyzed their cost in continuous integration.

B. MACHINE LEARNING FOR FLAKY TEST DETECTION

Applying machine learning (ML) to flaky test detection has gained traction. Ahmad et al. [1] compared classifiers like Random Forest (RF) and Decision Tree (DT) across languages, finding distinct performance patterns. Bell et al. [9] leveraged classification models to improve flaky test recall, focusing on reducing false negatives.

Rahman et al. [13] proposed FlakeSync, a tool employing ML to repair flaky tests. Fatima et al. [17] introduced Flakify, an NLP-based flaky test predictor, emphasizing textual and structural features. Miranda et al. [18] and Lin et al. [19] explored feature engineering, demonstrating the effectiveness of vocabulary-based classifiers.

C. CROSS-LANGUAGE AND CROSS-PROJECT GENERALIZATION

Flaky test detection in cross-language contexts has been extensively studied. Ahmad et al. [1] evaluated ML classifiers across Java, Python, C++, and Go, identifying patterns shared across languages. Similarly, Wendler et al. [20] analyzed dataset diversity's impact on classifier generalizability.

Cordy et al. [21] studied flaky tests across projects, revealing challenges in replicability and dataset dependency. Hilton et al. [6] demonstrated how cross-project data can enhance classifier robustness, complementing Wendler et al. [20] on using regression test history for flaky test prediction.

D. REPRODUCTION AND MITIGATION OF FLAKY TESTS

Reproducing flaky tests has been a persistent challenge. Gambi et al. [12] and Bell et al. [9] proposed tools like FlakeWatcher and DeFlaker for automating flaky test reproduction. Marinov et al. [16] emphasized detecting test dependencies for mitigating flaky behavior. Rahman et al. [13] introduced Shaker for automating flaky test repair.

E. FEATURE ANALYSIS AND BEHAVIORAL INSIGHTS

Static and dynamic features in flaky test detection have been extensively analyzed. Pinto et al. [14] and Camara et al. [15] identified critical features like asynchronous behaviors. Lin et al. [22] highlighted the significance of feature combinations, showing that dynamic features often outperform static ones.

Miranda et al. [18] demonstrated how feature sets impact model accuracy, aligning with Ahmad et al. [1], who highlighted trade-offs between feature complexity and classifier generalizability.

F. EMERGING TRENDS AND METHODOLOGICAL ADVANCES

Recent studies emphasize reproducibility and scalability. Wendler et al. [20] and Rahman et al. [13] advocate for standardized datasets to ensure consistency. Fatima et al. [17] explored advanced NLP techniques for flaky test detection.

Ahmad et al. [1] demonstrated how multi-language datasets enhance classifier robustness. Cordy et al. [21] and Marinov et al. [16] highlighted the interplay between language-specific and universal patterns.

III. RESEARCH METHODOLOGY

This study aimed to present methodology that uses machine learning techniques to identify and categorize flaky test cases across multiple programming languages. To identify flaky and non-flaky test cases, the methodology comprised preprocessing, feature extraction, systematic data collection, and the use of multiple classifiers. Each step of the procedure is covered in detail in this section, along with explanations of the methods and classifiers selected based on earlier studies.

The steps for flaky test detection across various programming languages are depicted in this process flow diagram, which is displayed in Fig 1. First, a pre-existing flaky dataset is combined with test cases for C++, JavaScript, Go, Java, and Python that are mined from GitHub. To find flaky tests, test cases are rigorously executed and run 100 times. Following identification, the test cases are preprocessed by eliminating symbols and stopwords, splitting CamelCase, and deleting comments. The data is made ready for tokenization and additional analysis through this cleaning procedure. Following preprocessing, the data is fed into machine learning classifiers such as Random Forest, Decision Tree, Naive Bayes, and others after being split into training and testing sets, typically using a 70-30, 80-20, and cross-validation split. Lastly, these classifiers produce evaluation metrics like accuracy, confusion matrix, and F1-score, which aid in determining how reliable each classifier is at identifying flaky tests.

A. DATA COLLECTION

The data for this research was obtained from *GitHub* repositories and a pre-existing *flaky test dataset*. In order to guarantee robustness and diversity, we extracted test cases from projects in *C++*, *JavaScript*, *Go*, *Java*, and *Python*. We employed tools such as *GHTorrent* and *Sourcerer*, which are frequently employed for mining large-scale repositories [9]. In their research on flaky test detection, [9] proposed that the process of identifying flaky tests involved observing inconsistent behavior (intermittent pass/fail results) and rerunning each test case 100 times.

TABLE 1. Collected data for flaky and non-flaky test cases across five programming languages.

Language	Flaky	Non-Flaky
Java	851	1053
Python	113	97
C++	20	58
Go	20	70
JavaScript	18	43

In his book on *case study research*, the authors in [23] underscores the importance of employing a structured methodology and utilizing multiple data sources, particularly in exploratory research that examines contemporary, real-world phenomena. This methodology was replicated in our investigation, which capitalized on both pre-existing datasets and mined repositories. We were able to ensure that the model was trained on test cases that had already been labeled by experts by utilizing the flaky test dataset, which consisted of labeled test cases for *Java* and *Python*. Consequently, we were able to establish a reliable baseline for the evaluation of classifier performance.

The dataset in Table 1 utilized in this study is composed of flaky and non-flaky test cases from five programming languages: *Java*, *Python*, *C++*, *Go*, and *JavaScript*. The majority of the dataset is derived from *Java*, with a total of 851 flaky tests and 1,053 non-flaky tests. Python follows with 113 flaky tests and 97 non-flaky tests. *C++* and *Go* are two languages that have 20 flaky test cases each, as well as 58 and 70 non-flaky tests, respectively. Additionally, the dataset in Table 1 comprises 18 flaky and 43 non-flaky tests from *JavaScript*. The extensive use of the language in the software development industry may be reflected in the dominance of *Java* in our dataset, as evidenced by these distributions. In contrast, languages such as *JavaScript*, *Go*, and *C++* contribute fewer test cases; however, they are still crucial for assessing the model's generalizability across various programming languages.

B. DATA PREPROCESSING

In order to convert raw test cases into a format appropriate for machine learning, preprocessing was essential. A number of important methods were used, such as tokenization, *CamelCase splitting*, comment removal, and the elimination of special symbols and stop words. In order to minimize noise and guarantee that only significant code elements were utilized during the classification process, the following actions were taken:

- **Removing Comments and Splitting CamelCase**: Comment removal guaranteed that the classification process was not influenced by non-functional, irrelevant

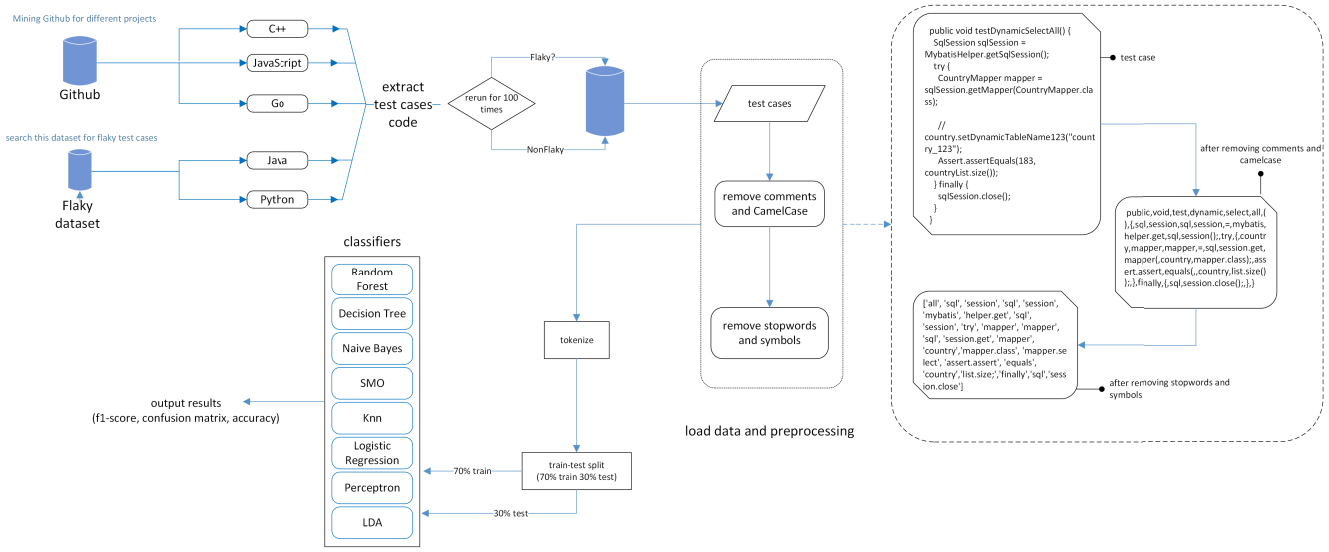


FIGURE 1. Data collection, analysis, Processing and validation in the Study.

information, while CamelCase splitting enhanced token granularity by deconstructing concatenated function or variable names. This method is frequently employed in *natural language processing (NLP)* for source code analysis, as recommended by [24]. They recommend that code structure be deconstructed into more precise tokens to enhance the accuracy of code analysis tasks.

- **Tokenization**: The test cases were divided into discrete tokens using tokenization, which divided each code element into more manageable chunks like operators, identifiers, and keywords. This step is in line with the methodology outlined by [25], in which tokenization serves as the basis for the majority of NLP text-processing tasks, enabling improved data representation in machine learning models.
- **Stop Word Removal and Symbol Filtering**: We eliminated common stop words and special symbols that didn't provide useful information for classification in order to reduce noise in the dataset. The decision to eliminate these components adheres to the rules outlined in [25], wherein superfluous terms (like keywords and operators that show up in every test case) were recognized as noise and eliminated.

C. FEATURE EXTRACTION AND DATA REPRESENTATION

To prepare the tokenized data for machine learning, we used the *term frequency-inverse document frequency (TF-IDF)* method. TF-IDF is a well-established technique for converting text data into numeric vectors by weighting the importance of each token relative to its frequency of appearance in the dataset [26]. This technique is widely used in NLP tasks because it emphasizes the most informative tokens while minimizing common but meaningless ones. Several studies, including [27], have demonstrated TF-IDF's effectiveness in text classification tasks.

D. CLASSIFIER SELECTION AND JUSTIFICATION

The study's critical component was the selection of classifiers for flaky test classification. Several classifiers that are frequently employed in text and code analysis were selected for their unique capabilities in managing structured data. We used the classifiers already studied in [14] and [15]

- **Random Forest**: Random Forest was chosen because it can handle complex, high-dimensional data by combining predictions from multiple decision trees. According to [28], Random Forest reduces overfitting by averaging results from multiple trees. This makes it ideal for non-linear or complex patterns, such as flaky test classification. Random Forest's ensemble nature improves generalization, making it one of the best options for classification tasks with diverse feature sets.
- **Decision Tree**: A Decision Tree classifier was implemented due to its interpretability and simplicity. Decision trees can be especially beneficial for comprehending the impact of specific features (e.g., function calls or external dependencies) on flaky test behavior. Decision trees are an effective tool for comprehending flaky test behavior when the decision boundaries are straightforward and comprehensible, as per [29].
- **Naive Bayes**: Assuming feature independence, the probabilistic classifier Naive Bayes, which is based on Bayes' Theorem, performs especially well in high-dimensional spaces. Because tokens taken from test cases can be regarded as independent features, it is appropriate for our study. Numerous studies have acknowledged the effectiveness and ease of use of Naive Bayes in text classification tasks, such as the

work by [30], which demonstrates that Naive Bayes performs surprisingly well even in situations where its independence assumption is not met.

- **Logistic Regression**: Using a linear combination of input data, logistic regression was selected for its capacity to model binary outcomes—flaky against non-flaky. Interpretability of logistic regression is well-known; it helps one understand how every feature influences the likelihood of a test case flaky. A common choice for many classification tasks, particularly in cases where the relationship between features and outcomes is rather linear, [31] proved the strength of logistic regression in binary classification problems.
- **Support Vector Machine (SVM)**: High-dimensional data handling and decision boundary creation with maximum margin between the two classes—flaky and non-flaky—led SVM to be selected. Since SVM can apply kernel functions to generate non-linear decision boundaries, [32] demonstrated that it is efficient in circumstances whereby the data is not linearly separable. This makes handling difficult patterns in flaky test data especially helpful.
- **K-Nearest Neighbors (KNN)**: KNN is a straightforward yet efficient algorithm that categorizes test cases according to the majority class of their nearest neighbors. KNN can be highly effective in identifying clusters of flaky behavior when there is sufficient data to determine neighborhood structure, as emphasized in [33].
- **Perceptron**: A basic linear classifier, the Perceptron modifies weights in response to classification errors. The Perceptron is useful for handling linearly separable data because it is easy to use and efficient, even though it is not as strong as more recent classifiers. Reference [34] presented the Perceptron, showcasing its effectiveness in binary classification tasks. While more advanced models have since overtaken it, it still offers a solid foundation for linear classification.
- **Linear Discriminant Analysis (LDA)**: LDA makes the assumptions that each class has a Gaussian distribution and that the classes (flaky and non-flaky) are linearly separable. This method is a good fit for tasks where the features are dispersed in clear patterns since it performs well when the data for each class can be represented by a linear decision boundary. LDA was initially presented by [35], who demonstrated its value in classifying classes with distinct means and variances.

E. HYPER PARAMETERS

We employ the same parameter settings as in [15] to assess the performance of various ML algorithms across a variety of programming languages and to compare our findings with those of previous studies. In our experiments, we employ eight classifiers that are implemented with Scikit-learn. The default parameter settings are implemented for the Naive Bayes and Linear Discriminant Analysis classifiers. The *CalibratedClassifierCV()* is employed to estimate and calibrate the predicted probabilities of the SVM and Perception classifiers. The parameters of our experiment are illustrated in Listing 1.

```
"random_forest": RandomForestClassifier(
  random_state=1)
"decision_tree": DecisionTreeClassifier(
  min_samples_leaf=1)
"naive_bayes": GaussianNB()
"svm": CalibratedClassifierCV(LinearSVC(
  fit_intercept=False, tol=0.001, C=1, dual=
  False, max_iter=100000), method="sigmoid")
"knn": KNeighborsClassifier(n_neighbors=1,
  metric="euclidean")
"LR": LogisticRegression(max_iter=1000)
"perception": CalibratedClassifierCV(Perceptron
  ())
"LDA": LinearDiscriminantAnalysis()
```

Listing 1. The parameters of our experiment.

F. TRAINING AND EVALUATION

The dataset was split into *70% training data*, *30% test data* and *10-fold CV*. Each model was trained on the training data and evaluated on the test set using metrics such as *accuracy*, *F1-score*, and the *confusion matrix*.

G. EVALUATION METRICS: PRECISION, RECALL, F1-SCORE, AND ACCURACY

We used precision, recall, F1-score, and accuracy—four widely used evaluation metrics—to assess how well our classifiers performed in identifying flaky tests. These metrics offer a comprehensive picture of the classifiers' performance in identifying flaky tests, weighing false positives and false negatives against overall correctness. These metrics have been widely used in comparable studies and are well-established in the software testing community. Although other metrics like ROC-AUC, MCC, and Cohen's Kappa might offer more information, they weren't employed in this study since the metrics that were selected offered an adequate balance between false positives, false negatives, and overall classification performance.

1) PRECISION

The precision calculates the percentage of all predicted positives that are true positives. Precision aids in determining the proportion of test cases that are truly flaky in the context of flaky test detection. This is essential for lowering false positive rates, which can result in needless debugging effort by misclassifying non-flaky tests as flaky. When the cost of a false positive is high, as in flaky test detection, where developers might have to spend time looking into falsely identified flaky tests, precision is crucial. Prior research has highlighted the importance of accuracy in these situations, where incorrect classification can have expensive consequences for developers and testers [36].

2) RECALL

The classifier's capacity to identify flaky tests that are truly flaky, as measured by the proportion of true positives out of all

actual positives. This metric is essential for comprehending the model's ability to identify flaky tests without omitting a significant number of them. This is particularly relevant in situations where the failure to detect flaky tests may result in undetected bugs or intermittent test failures [37]. Recall is instrumental in the detection of flaky tests, as it ensures that the majority of flaky tests are detected, thereby reducing the probability of flaky tests compromising the codebase's reliability. If flaky tests are not identified and addressed, they may obscure genuine failures and result in a decline in confidence in the test suite.

3) THE F1-SCORE

The F1-Score offers a harmonic mean between precision and recall, so balancing the two. F1-Score provides a whole metric that helps balance the trade-off between false positives and false negatives by combining accuracy and recall. This statistic is especially helpful in situations like flaky test detection when both recall and accuracy are critical [38]. In our work, F1-score guarantees that both misclassified non-flaky tests (false positives) and missed flaky tests (false negatives) are equally considered, so offering a balanced assessment of classifier performance.

4) ACCURACY

The accuracy is the general percentage of all cases—positive and negative—that are properly classified. As is the case in flaky test detection where non-flaky tests may greatly outnumber flaky tests [39], accuracy can be misleading in cases when the data is imbalanced. Accuracy still offers a useful overall assessment of classifier performance across several programming languages and test cases, though, given that we ran tests involving many classifiers and combinations of datasets.

5) ALTERNATIVE METRICS NOT USED

While Precision, Recall, F1-Score, and Accuracy provided a comprehensive understanding of the classifiers' performance, there are other metrics that could have been used but were not included in our study.

6) ROC-AUC (RECEIVER OPERATING CHARACTERISTIC - AREA UNDER CURVE)

For binary classifiers, ROC-AUC is a helpful metric that gives an overall assessment of classifier performance across all classification thresholds. Although this metric might have been helpful, we decided not to use it because our goal was to assess classification performance at a particular threshold, and in these situations, precision, recall, and F1-score provide more useful information [40].

7) MATTHEWS CORRELATION COEFFICIENT (MCC)

Even for datasets that are unbalanced, MCC offers a fair assessment by taking into account both true and false positives as well as negatives. But in practice, MCC is frequently harder to understand than F1-Score and accuracy, which provide simpler information about the precision-recall trade-off [41].

H. STATISTICAL ANALYSIS

We conducted paired t-tests on accuracy and F1-scores to determine the importance of performance differences among classifiers. Random Forest, Decision Tree, Naive Bayes, k-Nearest Neighbors (kNN), Support Vector Machine (SVM), Perceptron, Logistic Regression, and Linear Discriminant Analysis (LDA) comprised the classifiers. The results of each classifier pair were compared across various programming languages (Java, Python, Go, C++) and their combinations. The objective of the t-tests was to detect statistically significant differences ($p < 0.05$) in the performance of the classifiers, with an emphasis on the F1-score and accuracy as critical metrics. This statistical method enhances the evaluation's rigor by guaranteeing that the observed performance trends are not the result of random variation.

I. EXPERIMENT 1: INDIVIDUAL LANGUAGE CLASSIFICATION (RQ1.1 - RQ 1.4)

We applied each classifier to test cases from each programming language separately in the initial experiment. This method enabled us to assess the classifiers' performance in various linguistic contexts and comprehend the potential impact of language-specific characteristics on the identification of flaky tests. Random Forest (RF), Decision Tree (DT), Naive Bayes (NB), Support Vector Machine (SVM), K-Nearest Neighbors (KNN), Perceptron, Logistic Regression (LR), and Linear Discriminant Analysis (LDA) comprised the classifiers. The performance of each classifier was assessed using the following metrics: precision, recall, F1 score, and accuracy.

J. EXPERIMENT 2: CROSS-LANGUAGE PAIRING AND MULTI-LANGUAGE CLASSIFICATION (RQ2)

In order to determine whether combining linguistic contexts can improve classification performance, we conducted cross-language experiments in the second stage by pairing different programming languages in different combinations. Datasets combining test cases from two or more languages were created using pairings like *Java* with *Python*, *Java* with *C++*, *Java, Python, and C++*, and so on. This method enables us to investigate whether combining linguistic data can enhance classifier generalization and how combining test cases from various languages impacts flaky test prediction.

This cross-language experimentation is motivated by the hypothesis that when classifiers are exposed to a wider variety of test cases, they may become more adept at identifying flaky tests due to shared structural or functional features between languages, such as object-oriented paradigms (found in Python, Java, and C++, for example). Additionally, cross-language learning could improve the robustness of the classifiers by identifying common patterns or behaviors of flaky tests that are not dependent on particular syntactical details.

K. EXPERIMENT 3: COMPARISON WITH PREVIOUS STUDIES (RQ3)

Apart from our initial experiments, we expanded our investigation by contrasting our findings with two earlier works: "What is the Vocabulary of Flaky Tests? [14]" and its replication study, "What is the Vocabulary of Flaky Tests? "A Comprehensive Interpretation" [15]. With an emphasis on the vocabulary of flaky tests, these two studies also investigated flaky test detection using a variety of machine learning techniques. We compared them using various metrics and representations to determine which had the highest or lowest predictive accuracy.

L. RESEARCH METHOD SUMMARY

The objective of the investigation was to assess the efficacy of machine learning classifiers in identifying flaky tests in a variety of programming languages, as delineated in Table 2. The study aims to investigate the diverse effects of cross-language and language-specific features on the performance of classifiers. The research context is centered on the detection of flaky tests in open-source software repositories, which are representative of real-world scenarios of automated testing in a variety of programming environments. The study includes programming languages such as Java, Python, C++, Go, and JavaScript, which were selected for their widespread use and diverse testing ecosystems. The theoretical foundation is predicated on the hypothesis that the performance of classifiers is influenced by both shared features across languages and unique language-specific patterns. The investigation is guided by research questions (RQ1.1 through RQ3), which are designed to assess the impact, generalizability, and reliability of various classifiers. The techniques entailed the utilization of machine learning classifiers for detection, preprocessing test cases to extract meaningful patterns, and archival data collection from repositories. The selection strategy was designed to guarantee that there was an adequate amount of test data and diversity, which would facilitate a thorough analysis. The units of analysis are divided into individual test cases (flaky and non-flaky) and datasets that are grouped by language and cross-language contexts, thereby enabling a thorough comprehension of the challenges associated with detecting flaky tests.

IV. RESULTS

A. CLASSIFIER PERFORMANCE WITHIN INDIVIDUAL LANGUAGES - RQ 1.1

For each programming language, we assessed the classifiers' performance in isolation to observe language-specific variations in flaky test detection as presented in Table 3.

Java: In Java, the classifiers consistently demonstrated strong performance, with Random Forest, Naive Bayes, Nearest Neighbor, and Logistic Regression achieving high accuracy (accuracy of **0.96**). These findings indicate that classifiers are capable of accurately identifying flaky patterns in Java-based test cases.

C++: Random Forest, SVM, and Logistic Regression achieved an accuracy of **0.96**, while the majority of classifiers in C++ maintained high accuracy. The slightly lower performance of Naive Bayes (accuracy of **0.83**) and LDA (accuracy of **0.63**) implies that flaky patterns in C++ may be more difficult to identify when using probabilistic or linear separation approaches.

Go: Go yielded strong classifier results, with both Random Forest and Decision Tree achieving perfect scores across precision, recall, F1-score, and accuracy (**1.0** each). This result suggests that flaky tests in Go may exhibit unique or easily identifiable patterns.

Python: Naive Bayes achieved the highest values for precision, recall, and F1-score (**0.97** each), while Python exhibited similarly high accuracy across most classifiers. Nearest Neighbor also exhibited exceptional performance, achieving perfect scores (**1.0**) across all metrics, which suggests that it is well-suited to Python's flaky test characteristics. It is important to note that the prevalence of web-based testing in Python is underscored by feature terms such as *model*, *flask*, and *config*, which may have an impact on flaky behavior. The model's robust capacity to identify patterns is suggested by its ability to achieve an accuracy of 1.0 in Python, which is likely a result of the unique features of the dataset. Nevertheless, this finding necessitates precaution in order to exclude overfitting, data leakage, or dataset imbalance. Consistency is verified through additional validations; however, generalizability necessitates testing on datasets that have not been previously examined. These results emphasize the method's potential while also underscoring the necessity of meticulous interpretation. Additional information is available in Section VI-A.

B. MULTI-LANGUAGE AND CROSS-LANGUAGE PERFORMANCE - RQ 1.2

By combining Java and JavaScript, Java, Python, and C++, as well as a complete set of all available languages (Java, Python, C++, Go, and JavaScript), the study further investigated the classifiers' performance on multi-language datasets. This method assisted in determining whether exposing the classifiers to a variety of test cases from various languages could improve their capacity to identify flaky tests and generalize.

Java & JavaScript Combination: Combining Java and JavaScript, classifiers like Random Forest, Logistic Regression, and Nearest Neighbor achieved high accuracy (**0.96-0.97**) as presented in Table 3. This result suggests that Java and JavaScript share flaky test features like *throws*, *exception*, and *assert*. The cross-language generalization observed in this combination implies that classifiers can recognize flaky patterns in languages with similar paradigms.

Java, Python, and C++ Combination: Random Forest, SVM, and Nearest Neighbor achieved an accuracy of **0.96–0.97**, as presented in Table 3, when training on

TABLE 2. Case study planning according to guidelines presented by Runeson and Höst [42].

Objective	Explore the effectiveness of machine learning classifiers in detecting flaky tests across programming languages.
The context	Flaky test detection in automated testing across open-source repositories and programming ecosystems.
The cases	Java, Python, C++, Go, and JavaScript, representing diverse testing ecosystems.
Theory	Classifier performance is influenced by language-specific patterns and shared cross-language features.
Research questions	RQ1.1, RQ1.2, RQ1.3, RQ1.4, RQ2, and RQ3.
Methods	Archival data collection, test case preprocessing, and machine learning classification for flaky test detection.
Selection strategy	Selection of open-source repositories to ensure diversity and sufficient test data.
Unit of Analysis 1	Individual test cases: flaky and non-flaky.
Unit of Analysis 2	Datasets grouped by language and cross-language contexts.

a combination of Java, Python, and C++. Classifiers maintained strong performance. The robustness and generalizability of these classifiers across three languages that exhibit significant structural and syntax differences are illustrated by their exceptional performance. Cross-language flakiness factors associated with error handling and resource utilization are further indicated by critical features, such as *exception*, *output*, and *description*.

Go, Python, C++, and JavaScript Combination: Random Forest achieved an accuracy of **0.90** in the broadest combination of languages, while SVM, Decision Tree, and Naive Bayes achieved moderate scores of **0.86**. The classifiers exhibited slight variability in this context. This finding suggests that, although diverse language inputs can improve generalization, they may also introduce complexities that decrease the precision of classifiers. Variability in feature types may complicate the detection of flaky tests across disparate languages, as evidenced by terms such as *string*, *func*, and *client*.

Classifiers like Random Forest, SVM, and Logistic Regression demonstrated high overall accuracy (**0.95**), as presented in Table 3, when tested and trained on a comprehensive dataset that included all languages (Java, Python, C++, Go, and JavaScript). Regardless of language-specific features, this impressive performance shows that classifiers trained on a variety of language sets can still generalize well and detect erratic patterns. *assert*, *resource*, and *output* were significant features in this multilingual context, indicating that flaky tests may frequently be related to resource dependencies and error assertions, which are shared by all languages.

The robust choices for flaky test detection across different languages are Random Forest (RF) and k-Nearest Neighbors (kNN), as demonstrated by the expanded accuracy heatmap in Figure 2. These models consistently achieve high accuracy, particularly in languages such as Go and Python. In contrast, Linear Discriminant Analysis (LDA) exhibits lower accuracy across the majority of languages, which is likely a result of its limitations with non-linear, high-dimensional data. Furthermore, the accuracy of the majority of classifiers is reduced by combined languages (e.g., ''Java & Python & C++''), which implies that mixed-language data introduces complexities. In general, RF and kNN are the most adaptable classifiers for multi-language contexts, while classifiers that are sensitive to language-specific patterns, such as LDA, may necessitate additional tuning. These results emphasize the importance of generalizable, robust classifiers in the detection of flaky tests across multiple languages.

C. ANALYSIS OF TOP 10 FEATURES ACROSS DIFFERENT LANGUAGES - RQ 1.3

We found both common patterns and distinctive language-specific characteristics when we examined the top 10 characteristics that were most suggestive of flaky tests in various languages. These characteristics, which are highlighted in Table 3, shed light on the fundamental reasons why tests in Java, Python, C++, Go, and combined multi-language datasets are flaky.

The top features linked to flaky test detection across individual languages and language combinations are clearly visualized by this 3 × 3 grid of word clouds, as seen in Figure 3. Larger words indicate terms commonly found in flaky tests, and each word cloud highlights the most important keywords. For instance, words like ''assert,'' ''error,'' and ''exception'' are frequently used in multiple languages, suggesting that they all contribute to test instability. Additionally apparent are patterns specific to a particular language, such as ''flask'' and ''config'' in Python, which imply that flakiness in this language may be influenced by particular frameworks or configurations. Mixed-language clouds, such as ''Java & JS'' or ''Java & Python & C++,'' show overlapping terms like ''should'' and ''throws,'' suggesting that languages share flakiness sources. While distinct words in single-language clouds highlight particular language-related problems, the presence of similar features in multi-language combinations suggests that some flaky test indicators are universal. All things considered, this visualization provides information about both common and distinctive factors influencing test flakiness across languages.

1) LANGUAGE-SPECIFIC FEATURES

Despite these commonalities, several features were unique to particular languages, reflecting language-specific testing practices and idioms. For instance:

TABLE 3. Predictive accuracy of individual languages and combinations together with top 10 features.

Our study						
Language	**Clf**	**P**	**R**	**F1**	**Acc**	**Top 20**
Java	Random Forest	0.96	0.96	0.96	0.96	error, assert, resource,for, get, throws, exception, should, io, equal, int, session, description, output, expect, close, factory, url, inode, byte
	Decision Tree	0.95	0.95	0.95	0.95	
	Naive Bayes	0.97	0.97	0.96	0.96	
	SVM	0.96	0.95	0.95	0.95	
	Nearest Neighbour	0.96	0.96	0.96	0.96	
	Perceptron	0.94	0.94	0.94	0.94	
	LR	0.96	0.96	0.96	0.96	
	LDA	0.77	0.77	0.77	0.77	
Python	Random Forest	0.95	0.94	0.94	0.94	model, msgs, true, app, instance, flask, check, err, config,batch, models, benchmark, dict,false,equal, with, expected, assert, multi,https,
	Decision Tree	0.92	0.90	0.91	0.90	
	Naive Bayes	0.97	0.97	0.97	0.97	
	SVM	0.95	0.94	0.94	0.94	
	Nearest Neighbour	1.0	1.0	1.0	1.0	
	Perceptron	0.96	0.95	0.95	0.95	
	LR	0.95	0.94	0.94	0.94	
	LDA	0.79	0.79	0.79	0.79	
C++	Random Forest	0.96	0.96	0.96	0.96	case, check, string, expect, and, int, asset, disabled, float, time, make, require, in, wait, manager op, data, return, for, milliseconds,
	Decision Tree	0.92	0.0.92	0.90	0.92	
	Naive Bayes	0.92	0.83	0.85	0.83	
	SVM	0.96	0.96	0.96	0.96	
	Nearest Neighbour	0.96	0.96	0.96	0.96	
	Perceptron	0.92	0.92	0.90	0.92	
	LR	0.96	0.96	0.96	0.96	
	LDA	0.81	0.63	0.67	0.63	
Go	Random Forest	1.0	1.0	1.0	1.0	func, assert, pod, fake, file, string, sleep, poll, true, timeout, clock, if, time, no, client, err,esbuild, null, nil, error
	Decision Tree	1.0	1.0	1.0	1.0	
	Naive Bayes	0.96	0.96	0.96	0.96	
	SVM	0.91	0.89	0.90	0.89	
	Nearest Neighbour	0.96	0.96	0.96	0.96	
	Perceptron	0.96	0.96	0.96	0.96	
	LR	1.0	1.0	1.0	1.0	
	LDA	0.95	0.85	0.88	0.85	
Java JS	Random Forest	0.96	0.96	0.96	0.96	throws, exception, should, io, error, assert, resource, for, get, equal, int, session, description, output, expect, close, factory, url, inode, byte
	Decision Tree	0.95	0.95	0.95	0.95	
	Naive Bayes	0.95	0.95	0.95	0.95	
	SVM	0.95	0.95	0.95	0.95	
	Nearest Neighbour	0.97	0.97	0.97	0.97	
	Perceptron	0.94	0.94	0.94	0.94	
	LR	0.96	0.96	0.96	0.96	
	LDA	0.77	0.77	0.77	0.77	
Java Python C++	Random Forest	0.96	0.96	0.96	0.96	throws, should, exception, io, error, client, resource, for, output, close, that, get, description, thrown,message, equal, assert, string, inode, int
	Decision Tree	0.93	0.93	0.93	0.93	
	Naive Bayes	0.93	0.95	0.94	0.94	
	SVM	0.95	0.95	0.95	0.95	
	Nearest Neighbour	0.97	0.97	0.97	0.97	
	Perceptron	0.92	0.92	0.92	0.92	
	LR	0.96	0.96	0.96	0.96	
	LDA	0.73	0.73	0.73	0.73	
Go Python C++ JS	Random Forest	0.91	0.91	0.91	0.90	string, case, check, client, err, func, unit, true, equal, false, text, function, instance, flask, files, assert, context, app, append, model
	Decision Tree	0.86	0.86	0.86	0.86	
	Naive Bayes	0.86	0.86	0.86	0.86	
	SVM	0.86	0.86	0.86	0.86	
	Nearest Neighbour	0.94	0.94	0.94	0.94	
	Perceptron	0.93	0.93	0.92	0.92	
	LR	0.93	0.94	0.93	0.93	
	LDA	0.84	0.80	0.81	0.80	
All	Random Forest	0.95	0.94	0.95	0.95	throws, should, io, exception, client, error, assert, equal, factory, that, get, string, resource, output,inode, representation, for, description, table, int
	Decision Tree	0.93	0.93	0.93	0.93	
	Naive Bayes	0.92	0.91	0.92	0.92	
	SVM	0.95	0.95	0.95	0.95	
	Nearest Neighbour	0.98	0.96	0.97	0.97	
	Perceptron	0.93	0.93	0.93	0.93	
	LR	0.95	0.95	0.95	0.95	
	LDA	0.77	0.77	0.77	0.77	

- **Java:** The terms *resource*, *throws*, *for*, and *session* were among the most prominent features in Java. Java's structured resource management and exception handling, which requires explicit management and

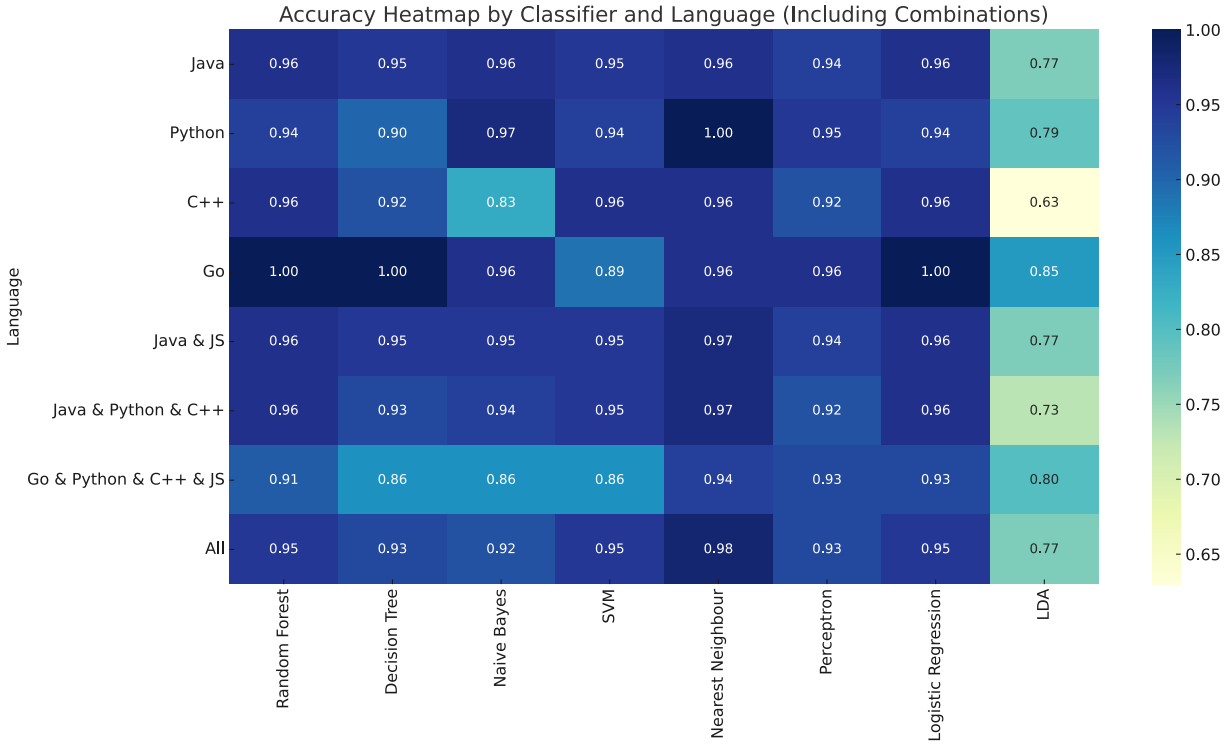


FIGURE 2. Accuracy HeatMap for classifier and languages.

release of resources like files or sessions, is reflected in these features. When exception handling is not robust, flaky tests may frequently involve functions that declare exceptions, as indicated by the presence of *throws*. This could result in inconsistent behavior.

- **Python:** Terms like *flask*, *config*, *model*, and *app* were among the Python-specific features. These terms show that testing web applications or configurations is frequently associated with flaky Python tests. For instance, *flask* and *app* imply a connection to the Flask web framework, which is recognized for tests that might be flaky because of configuration inconsistencies or dependencies on server states.
- **C++:** *string*, *float*, *wait*, and *make* were among the most popular C++ features. These characteristics most likely relate to low-level tasks like floating-point computations, strings, and memory management. Words like *wait* and *make* are frequently used in multi-threaded contexts or build processes, indicating that timing problems or race conditions in parallel test execution could be the cause of flakiness in C++ tests.
- **Go:** *func*, *timeout*, *clock*, and *client* were special features for Go. Go tests are especially sensitive to time-based dependencies, as indicated by the emphasis on *timeout* and *clock*. This is probably because Go is frequently used in applications that are focused on networks or concurrency. Go's functional programming style is reflected in the use of *func*, where handling errors within closures or passing around function pointers can lead to flakiness.

D. ANALYSIS OF VARYING TRAINING AND TESTING PERCENTAGES - RQ 1.4

We investigate the influence of various training-to-testing ratios (70-30, 80-20, and cross-validation) on the performance of classifiers in terms of precision (P) and recall (R) across individual languages (Java, Python, C++, Go) and combined datasets in this section.

Figure 4 demonstrates that K-Nearest Neighbors (kNN) is the consistent high performer across all metrics, particularly in the F1 score, where it even achieves a perfect score of 1.0 in some splits. The robust adaptability of kNN to the data distribution and its capacity to classify flaky tests with high precision and recall, particularly when trained on larger splits such as 80-20 and 70-30, are indicated by this high performance.

Although Random Forest (RF) remains a robust performer, it fails to achieve the same level of consistency as kNN in all splits and metrics. RF exhibits satisfactory performance; however, kNN's scores are either equivalent to or slightly superior, particularly in the areas of recall and F1 scores. This suggests that kNN may be a more dependable option for detecting flaky tests in this study, as it is capable of managing the data variability that is present across training splits.

When flaky tests are hard to find or don't happen very often, getting a precision of 1.0 means that all the cases

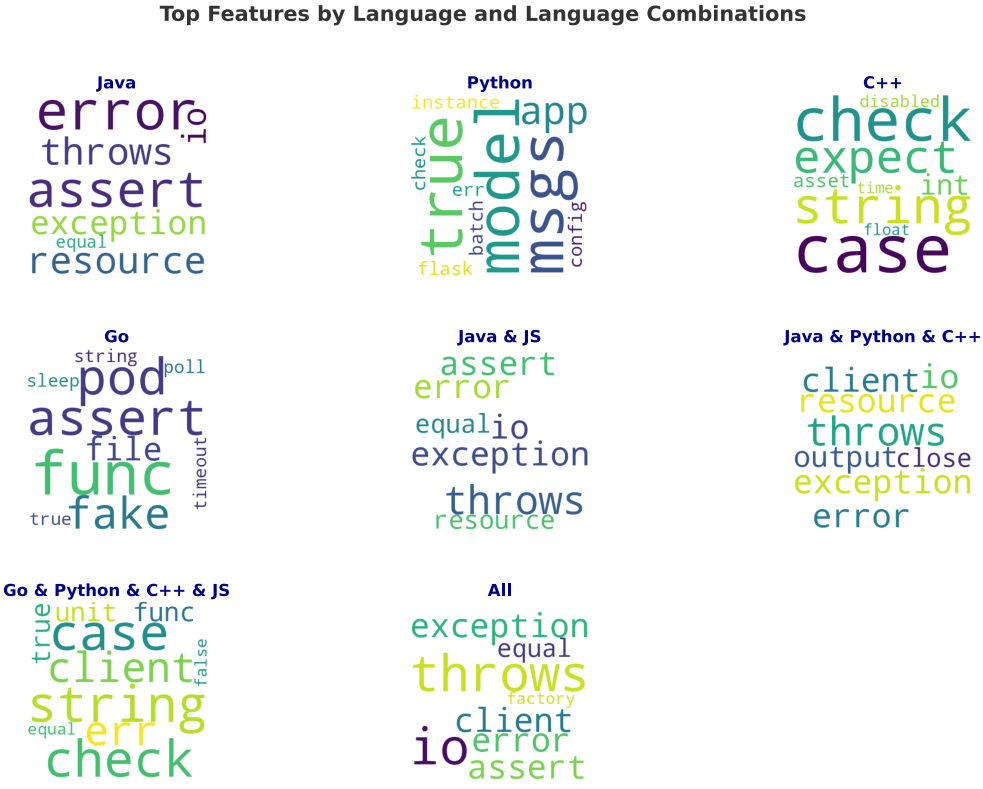


FIGURE 3. Separated top features WordClouds in all languages.

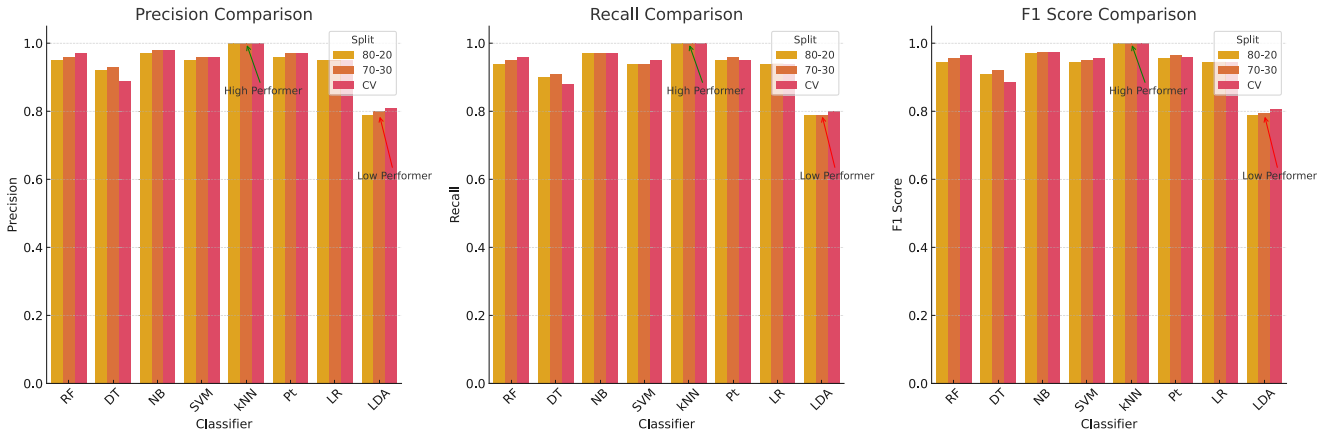


FIGURE 4. Comparison of classifiers based on different train and test split combinations.

that the kNN classifier marked as flaky were, in fact, flaky. This level of accuracy suggests that the model is very careful and does a good job of avoiding false positives. Being careful is very important in flaky test detection because false positives can cause developers to spend time and trust in automated testing systems on work that isn't necessary. The kNN classifier sorts things into groups based on how close they are to labeled instances in the feature space. kNN can be very accurate if the training data is well organized or strongly clusters around specific test behaviors, such as clear behaviors between flaky and non-flaky tests. This is because it will only classify instances that closely match clear flaky patterns. In real life, this could happen when flaky tests have special feature profiles that non-flaky tests don't have. This lets kNN avoid false positives well. Proof from other studies: Studies on flaky test detection, like Bell [9], show

that focusing on high precision is a good way to avoid wrong identification that could lead developers astray. This strategy works well with getting a high precision score of 1.0, which suggests that kNN might work well in places where false positives are very bad.

1) INDIVIDUAL LANGUAGE ANALYSIS

Java (J): The classifiers generally exhibit satisfactory precision and recall in all three configurations (70-30, 80-20, and cross-validation), with Random Forest (RF), Naive Bayes (NB), and Logistic Regression (LR) being particularly effective. For example, the precision/recall scores of RF are **0.96/0.95** in the 70-30 split and slightly increase to **0.97/0.94** in the 80-20 split. This indicates that the stability of the classifier is improved by increasing the training set. The cross-validation results indicate that the accuracy of classifiers such as LDA is reduced, with a value of **0.55/0.58**. This suggests that Java faces challenges with limited data diversity.

Python (P): Python's results show that classifiers like Naive Bayes, k-Nearest Neighbors (kNN), and Perceptron achieve perfect recall (**1.0**) under both splits, with high precision and recall scores in both the 70-30 and 80-20 splits. Classifiers like Random Forest and kNN work best with slightly larger training sets, according to the cross-validation results, which are still high but exhibit some variability. When training data is scarce, LDA's sensitivity to Python's data distribution is demonstrated by the performance drop (**0.83/0.72**).

C++ (C): Training splits affect the performance of the classifiers for C++ quite differently. For example, RF achieves **1.0/0.75** in the 70-30 split but the recall declines to **0.67** in the 80-20 split, so indicating decreased generalization as the size of the training set changes. With RF keeping perfect precision (**1.0**) and enhanced recall (**0.85**), cross-valuation results show a modest improvement in stability, implying that cross-valuation benefits classifiers that need data balancing, such C++' limited dataset.

Go (G): Classifiers such as RF, Decision Tree (DT), and LR demonstrate high precision and recall in Go data. RF achieves perfect precision and recall (**1.0/1.0**) in both the 70-30 and 80-20 splits. Nevertheless, certain classifiers, such as Naive Bayes and SVM, exhibit diminished performance during cross-validation. For example, Naive Bayes experiences a decrease in performance to **0.64/0.90**. This implies that Go's test cases may involve structural characteristics that are more appropriate for decision tree-based classifiers.

2) MULTI-LANGUAGE COMBINATIONS

Java and JavaScript (J&S): For this combination, classifiers perform consistently at various training ratios. High precision and recall are maintained by RF and kNN (e.g., **0.96/0.94** for RF in 70-30 split), suggesting that classifier robustness is strengthened when Java and JavaScript are combined. Cross-validation, however, shows a moderate decline, with LDA only reaching **0.58/0.59**. This implies that neighbor-based and decision-tree models are more adept than linear approaches at handling data variability in this combination.

Java, Python, and C++ (JPC): Classifiers such as RF and kNN maintain high scores in the 70-30 and 80-20 splits for the combination of Java, Python, and C++; in the 70-30 split, RF achieves **0.96/0.95**. According to cross-validation results, classifier precision and recall decline, especially for LDA, which falls to **0.63/0.59**. Since smaller samples make it difficult to generalize across languages with different syntactical structures, this trend suggests that mixed-language data helps classifiers with larger training sets.

Go, Python, C++, and JavaScript (GPCS): With scores of **0.87/0.90** and **0.90/0.94** in the 70-30 and 80-20 splits, respectively, the results show that RF and kNN are among the best. LDA performed the worst (**0.44/0.53**) in cross-validation results, indicating significant variability and challenges with multi-language diversity. This implies that linear classifiers suffer in the absence of adequate data continuity, whereas RF and kNN generalize well across complex datasets.

The results show that high accuracy can be attained by machine learning classifiers trained on individual languages, especially when working with languages like Python and Java where flaky test patterns are easier to spot. Classifiers such as Random Forest, SVM, and Nearest Neighbor demonstrated strong generalization skills when trained on multi-language datasets, suggesting that cross-language training can improve the detection of flaky tests. Language-independent elements that lead to test flakiness, like error handling, resource dependencies, and timeouts, are also highlighted by the cross-linguistic consistency of key feature terms.

According to this analysis, each language exhibits distinct patterns that most likely originate from its ecosystem, syntax, and common application domains, even though a core set of characteristics linked to test flakiness is shared by all languages. Language-specific terms improve model accuracy by capturing unique flaky behaviors unique to each language's use cases, while shared features (e.g., *assert*, *exception*) serve as a basis for cross-language classifiers. Given that classifiers can use both shared and language-specific features to adjust to various testing environments, these findings demonstrate how multi-language datasets may enhance generalization in flaky test detection.

The results indicate that:

- **Higher Training Ratios Enhance Stability:** Increasing the training set (80-20) generally improves precision and recall across classifiers, particularly for RF, kNN, and LR.
- **Cross-Validation Reveals Generalization Limits:** Cross-validation shows reduced classifier performance, especially for linear models like LDA, indicating challenges with smaller, diverse samples.

- **Decision-Tree and Neighbor-Based Models Perform Best:** RF and kNN consistently achieve high accuracy across all configurations and datasets, suggesting their robustness across languages and varied training-to-testing splits.

Due to their flexibility in managing a variety of datasets, these results indicate that decision-tree-based models are more appropriate for multi-language flaky test detection and emphasize the significance of balanced training ratios for enhancing classifier performance.

E. ANALYSIS OF CROSS-LANGUAGE CLASSIFIER PERFORMANCE - RQ 2

The results of cross-language training and testing are summarized in Table 4, presenting the F1-scores and accuracy for various classifiers across different train-test pairs of programming languages. This analysis reveals notable variations in classifier performance, highlighting how predictive accuracy is impacted when models trained on one language are applied to test cases in a different language.

1) KEY OBSERVATIONS BY CROSS-LANGUAGE PAIR

Java-Python: Overall performance of classifiers tested on Python and trained on Java was comparatively poor. Low F1-scores and accuracies were attained by the Random Forest (RF) and Decision Tree (DT) classifiers; RF's accuracy was **0.29** F1 and **0.40**, while DT's was **0.32** F1 and **0.37** as shown in Table 4. On the other hand, k-Nearest Neighbors (kNN) and logistic regression (LR) scored marginally higher, with LR attaining **0.66** F1 and **0.66** accuracy. This suggests that despite the structural and syntactical differences from Java, some classifiers, such as LR and kNN, might generalize better to Python data.

Java-C++: Compared to Java-Python, the classifiers performed better when trained on Java and evaluated on C++. For instance, LR obtained an accuracy of **0.79** and an F1-score of **0.81**. Another excellent performer in this setup, Random Forest, achieved **0.73** F1 and **0.78** accuracy. These findings imply that models trained in Java might be more suited to C++ test cases than Python, perhaps as a result of the two languages' similar object-oriented features and structural similarities.

Java-Go: Performance was moderate across classifiers when Java-trained classifiers were used on Go test cases; RF achieved **0.66** F1 and **0.71** accuracy, while DT achieved **0.67** F1 and **0.69** accuracy. This indicates that some classifiers maintain a respectable predictive power when cross-applying models from Java to Go, despite the results being weaker than those from Java-C++. This is probably because the procedural programming styles of Java and Go are similar.

Python-Java and Python-C++: When models were tested on Java or C++ after being trained on Python, the cross-language results were generally poor, especially for RF and DT, which had F1-scores of approximately **0.31–0.37**. Python's flexible typing and distinctive testing patterns, which are difficult to translate to statically typed languages like Java and C++, may be the cause of these low scores. With an F1-score of **0.49** and **0.50** accuracy in both configurations, the Naive Bayes (NB) classifier fared better, indicating that probabilistic methods like NB might provide some resilience when modifying Python-trained models.

C++-Java and C++-Python: Some classifiers showed moderate performance when tested on Python or Java after being trained on C++. In both configurations, for example, kNN obtained F1 and accuracy scores of **0.65/0.67**, and Perceptron (Pt) also demonstrated good performance, achieving an F1-score of **0.64**. These findings suggest that models developed in C++ may be able to generalize to other languages to some extent, possibly as a result of C++'s low-level features matching basic language operations.

Go-Python and Go-Java: The results of the training on Go and the testing on Python or Java were inconsistent. The F1-scores and accuracies of DT and kNN were in the **0.53–0.67** range, indicating that certain structural patterns in Go tests may be consistent with those in Python and Java. Consequently, their performance was reasonably high. Nevertheless, classifiers such as NB and RF exhibited inferior performance, which may be attributed to the concurrency-focused testing style and minimalistic syntax of Go, which are markedly different from those of Python and Java.

JavaScript (JS) Combinations: In the JS-Go and JS-C++ configurations, models that were trained on JavaScript and tested on other languages generally exhibited low predictive accuracy. For instance, when trained on JavaScript and tested on Go or C++, RF and DT both achieved an F1 of **0.56** and an accuracy of **0.56**. This performance discrepancy may be attributed to the event-driven, asynchronous test style of JavaScript, which is difficult to adapt to statically-typed languages or those lacking asynchronous patterns.

The analysis of cross-language training and testing reveals several key trends:

- **Classifiers Generalize Better Between Structurally Similar Languages:** Cross-language predictive accuracy is typically higher for languages like Java and C++ that share similar structures and paradigms. In these pairs, classifiers such as LR, RF, and kNN showed comparatively good performance, indicating that procedural structures and shared object-oriented features facilitate generalization.
- **Limited Predictive Accuracy for Dynamic and Asynchronous Languages:** When tested on statically-typed languages like Java and C++, classifiers trained on dynamic languages like Python and JavaScript perform poorly. This emphasizes how difficult it is to generalize across languages with disparate asynchronous patterns, testing procedures, and typing systems. In the Python-C++ and JS-Go configurations, for example, RF and DT scored noticeably poorly.

TABLE 4. Predictive accuracy of classifiers trained on one language and tested on another language.

tr/te	Clf	F1	Acc	tr/te	Clf	F1	Acc	tr/te	Clf	F1	Acc
	RF	0.29	0.40		RF	0.73	0.78		RF	0.66	0.71
	DT	0.32	0.37		DT	0.66	0.76		DT	0.67	0.69
	NB	0.49	0.50		NB	0.63	0.60		NB	0.51	0.47
	SVM	0.63	0.63		SVM	0.69	0.67		SVM	0.53	0.49
JP	kNN	0.65	0.66	JC	kNN	0.70	0.72	JG	kNN	0.61	0.60
	LR	0.66	0.66		LR	0.81	0.79		LR	0.57	0.53
	Pt	0.46	0.51		Pt	0.74	0.73		Pt	0.58	0.57
	LDA	0.50	0.52		LDA	0.18	0.24		LDA	0.57	0.54
	RF	0.57	0.67		RF	0.31	0.47		RF	0.17	0.33
	DT	0.56	0.62		DT	0.32	0.37		DT	0.25	0.27
	NB	0.49	0.51		NB	0.49	0.51		NB	0.49	0.50
	SVM	0.54	0.52		SVM	0.35	0.49		SVM	0.23	0.37
JS	kNN	0.59	0.62	PJ	kNN	0.31	0.43	PC	kNN	0.14	0.27
	LR	0.63	0.67		LR	0.33	0.48		LR	0.23	0.37
	Pt	0.56	0.62		Pt	0.29	0.46		Pt	0.17	0.33
	LDA	0.50	0.51		LDA	0.52	0.52		LDA	0.54	0.53
	RF	0.17	0.33		RF	0.17	0.33		RF	0.37	0.39
	DT	0.25	0.27		DT	0.25	0.27		DT	0.14	0.30
	NB	0.49	0.50		NB	0.49	0.50		NB	0.26	0.36
	SVM	0.23	0.37		SVM	0.23	0.37		SVM	0.59	0.58
PG	kNN	0.14	0.27	PS	kNN	0.14	0.27	CJ	kNN	0.65	0.67
	LR	0.23	0.37		LR	0.23	0.37		LR	0.45	0.48
	Pt	0.17	0.33		Pt	0.17	0.33		Pt	0.64	0.64
	LDA	0.54	0.53		LDA	0.54	0.53		LDA	0.31	0.39
	RF	0.37	0.39		RF	0.37	0.39		RF	0.37	0.39
	DT	0.14	0.30		DT	0.14	0.30		DT	0.14	0.30
	NB	0.26	0.36		NB	0.26	0.36		NB	0.26	0.36
	SVM	0.59	0.58		SVM	0.59	0.58		SVM	0.59	0.58
CP	kNN	0.65	0.67	CG	kNN	0.65	0.67	CS	kNN	0.65	0.67
	LR	0.45	0.48		LR	0.45	0.48		LR	0.45	0.48
	Pt	0.64	0.64		Pt	0.64	0.64		Pt	0.64	0.64
	LDA	0.31	0.39		LDA	0.31	0.39		LDA	0.31	0.39
	RF	0.47	0.49		RF	0.47	0.49		RF	0.47	0.49
	DT	0.53	0.67		DT	0.53	0.67		DT	0.53	0.67
	NB	0.48	0.56		NB	0.48	0.56		NB	0.48	0.56
	SVM	0.53	0.67		SVM	0.53	0.67		SVM	0.53	0.67
GJ	kNN	0.58	0.56	GP	kNN	0.58	0.56	GC	kNN	0.58	0.56
	LR	0.53	0.51		LR	0.53	0.51		LR	0.53	0.51
	Pt	0.53	0.67		Pt	0.53	0.67		Pt	0.53	0.67
	LDA	0.55	0.54		LDA	0.55	0.54		LDA	0.55	0.54
	RF	0.47	0.49		RF	0.56	0.56		RF	0.56	0.56
	DT	0.53	0.67		DT	0.17	0.30		DT	0.22	0.30
	NB	0.48	0.56		NB	0.57	0.67		NB	0.57	0.67
	SVM	0.53	0.67		SVM	0.54	0.63		SVM	0.54	0.63
GS	kNN	0.58	0.56	SJ	kNN	0.42	0.44	SP	kNN	0.42	0.44
	LR	0.53	0.51		LR	0.44	0.44		LR	0.44	0.44
	Pt	0.53	0.67		Pt	0.63	0.70		Pt	0.63	0.70
	LDA	0.55	0.54		LDA	0.32	0.41		LDA	0.32	0.41
	RF	0.56	0.56		RF	0.56	0.56				
	DT	0.17	0.30		DT	0.22	0.30				
	NB	0.57	0.67		NB	0.57	0.67				
	SVM	0.54	0.63		SVM	0.54	0.63				
SC	kNN	0.42	0.44	SG	kNN	0.42	0.44				
	LR	0.44	0.44		LR	0.44	0.44				
	Pt	0.63	0.70		Pt	0.63	0.70				
	LDA	0.32	0.41		LDA	0.32	0.41				

- **Certain Classifiers Show Robustness Across Diverse Languages:** The most consistent performance across different language pairs was shown by Logistic Regression (LR), kNN, and Naive Bayes (NB), indicating that these classifiers have greater cross-language adaptability. But in cross-language configurations, classifiers that depend on structural consistency—like Random Forest and Decision Tree—exhibited a more pronounced decline in performance.

In conclusion, depending on the structural and syntactical similarities between the languages in question, cross-language predictive accuracy for flaky test detection varies greatly. While models trained on languages with distinct testing patterns, like Python and JavaScript, have difficulty adapting to other languages, classifiers typically generalize better between languages with shared paradigms (e.g., Java and C++). This realization emphasizes how crucial it is to choose training data and classifiers carefully according to language properties in order to enhance cross-language generalization in flaky test detection.

F. COMPARATIVE ANALYSIS OF CLASSIFIER PERFORMANCE WITH PRIOR WORK - RQ 3

Table 5 provides a comparison between the original study (ORI) by Pinto et al. [14], the replication study (REP) by

TABLE 5. Comparison of current work with previous works.

	Ori			Rep			Our			Ori-Our			Rep-Our		
clf	P	R	F_1	P	R	F_1	P	R	F_1	P	R	F_1	P	R	F_1
RF	0.99	0.91	0.95	0.98	0.89	0.94	0.95	0.94	0.94	↘	↗	↘	↘	↗	≡
DT	0.89	0.88	0.89	0.87	0.86	0.86	0.92	0.90	0.91	↗	↗	↗	↗	↗	↗
NB	0.93	0.80	0.86	0.95	0.84	0.89	0.97	0.97	0.97	↗	↗	↗	↗	↗	↗
SVM	0.93	0.92	0.93	0.93	0.86	0.90	0.95	0.94	0.94	↗	↗	↗	↗	↗	↗
kNN	0.97	0.88	0.92	0.98	0.81	0.89	1.0	1.0	1.0	↗	↗	↗	↗	↗	↗
Pt				0.95	0.83	0.88	0.96	0.95	0.95				↗	↗	↗
LR				0.91	0.91	0.91	0.95	0.94	0.94				↗	↗	↗
LDA				0.83	0.78	0.80	0.79	0.79	0.79				↘	↗	↘

Camara et al. [15], and our study (OUR). Each classifier's performance in terms of precision (P), recall (R), and F1-score (F_1) is presented, with arrows indicating comparative changes between the studies. This comparison highlights which classifiers demonstrated improved performance in OUR study relative to prior work, as well as specific metrics that have seen gains or declines.

Random Forest (RF): Precision (**0.99**), recall (**0.91**), and F1-score (**0.95**) were all high in ORI for the RF classifier. With precision at **0.98**, recall at **0.89**, and F1-score at **0.94**, REP saw a minor decline. Our study's F1-score was stable at **0.94**, with recall improving to **0.94** and precision slightly lower at **0.95** as shown in Table 5. Although there was a slight trade-off in precision compared to ORI, the increase in recall (↗) from both earlier studies indicates that our RF model improves the detection of flaky test cases.

Decision Tree (DT): DT performance consistently improved across all three studies. ORI recorded an F1-score of **0.89**, while REP observed a slight decrease to **0.86**. OUR study showed further improvement, with precision at **0.92**, recall at **0.90**, and F1-score at **0.91**. These gains across all metrics (↗) indicate that DT in our study is more accurate and balanced, effectively capturing flaky test patterns with improved stability.

Naive Bayes (NB): In every study, NB showed steady improvements. It obtained an F1-score of **0.86**, recall of **0.80**, and precision of **0.93** in ORI. A slight increase in REP was noted, with recall reaching **0.84**. NB obtained the highest precision, recall, and F1-score in our study, all of which were at **0.97**. The substantial rise in all metrics (↗) indicates that NB in our study has good generalization skills and can detect flaky tests more reliably than in other studies.

Support Vector Machine (SVM): The F1-score for SVM performance in ORI was **0.93**, indicating balance. REP had a lower F1-score of **0.90** due to a slight decline in recall (**0.86**). SVM demonstrated improvements in our study, achieving a stable F1-score of **0.94** for both precision and recall. The overall increase (↗) across all metrics suggests that SVM offers improved flaky test detection and strong consistency in our study.

k-Nearest Neighbors (kNN): With ORI reporting an F1-score of **0.92** and REP observing a slight decline to **0.89**, kNN maintained good performance across all studies. Precision and recall were both at **1.0**, and our study showed a perfect F1-score of **1.0**. These results indicate that kNN performs exceptionally well in identifying flaky test cases with high accuracy and consistency in our study, as evidenced by a significant increase in performance metrics (↗).

Perceptron (Pt): Perceptron was introduced in the REP study, which reported an F1-score of **0.88**. All metrics improved in OUR study, with precision, recall, and F1-score all reaching **0.95**. In our study, Perceptron outperformed REP and showed improved flaky test identification, as evidenced by the increase in all metrics (↗).

Logistic Regression (LR): According to REP, LR's F1-score was **0.91**. Our study's F1-score was **0.94**, with precision and recall rising to **0.95** and **0.94**, respectively. The enhancement (↗) indicates that LR has profited from the methodological improvements made in our study, which has increased the detection accuracy.

Linear Discriminant Analysis (LDA): LDA demonstrated F1-score at **0.80**, precision at **0.83**, and recall at **0.78** in REP. In our study, precision, recall, and F1-score were all at **0.79**, indicating a slight decline in precision and F1-score. The drop in LDA (↘) performance indicates that it might have trouble generalizing, either because of the wider variety of faulty test cases or because of the variability in the features of our study.

The line chart in Figure 5 compares precision, recall, and F1-scores for various classifiers (RF, DT, NB, SVM, kNN, Pt, LR, and LDA) in the Original, Replication, and Our Study. Each line corresponds to one of the three studies, allowing us to compare the performance of each classifier on the metrics. Notably, kNN has the highest precision, recall, and F1-scores in all studies, implying that it may be especially effective in flaky test detection. Random Forest (RF) also consistently produces high metric values across studies, confirming its status as a top performer. On the other hand, LDA has lower scores, particularly in the Replication and Our studies, implying that it may not be as effective for this task. Naive Bayes (NB) and Support Vector Machine (SVM) performance improves in our study compared to the original, implying that our methods are more effective. This visualization aids in identifying classifiers that have proven to be reliable across studies and those that may require adjustments for improved performance.

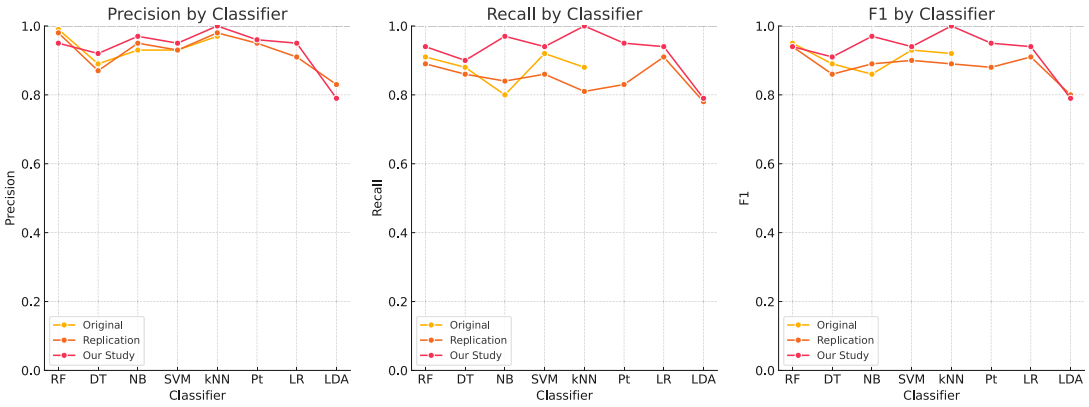


FIGURE 5. Comparison among different studies and classifiers.

The comparison between our study and previous work by Pinto et al. and Camara et al. reveals key insights:

- **Overall Performance Improvements in Our Study:** Our study's classifiers, especially DT, NB, SVM, and kNN, showed gains in recall, precision, and F1-score compared to earlier research. These enhancements suggest that the classifiers' capacity to identify flaky tests may have been improved by our feature selection, dataset composition, and experimental design.
- **Consistency in Classifier Performance Trends:** Similar trends across studies suggest that certain classifiers, like NB and kNN, consistently perform well in detecting flaky tests, while models like LDA may struggle with generalization.
- **Notable Gains in Recall:** Many classifiers, including RF, DT, NB, and SVM, demonstrated improved recall in our study, indicating better detection rates for flaky tests. This improvement in recall highlights the robustness of our model in minimizing missed flaky cases.
- **Enhanced Precision in Our Study:** Both kNN and NB demonstrated a high degree of accuracy in detecting genuine flaky tests, achieving nearly perfect precision. Compared to previous research, this improvement in precision implies that our classifiers are more adept at differentiating between flaky and non-flaky tests.

In summary, the classifiers in our study exhibit notable gains over those in previous research, especially with regard to recall and precision. These findings support the efficacy of our approach when compared to earlier research by indicating that our methodology and experimental improvements lead to more precise and reliable flaky test detection.

V. DISCUSSION

This discussion places our results within the larger body of literature on flaky test detection and interprets them in light of the research questions. This section offers insights into the effectiveness of machine learning models for flaky test detection across multiple programming languages, cross-language adaptability, and the relative advantages of our methodology by looking at the implications of each research question (RQ) and citing pertinent previous work. We also discuss the limitations and suggest directions for further study.

A. RQ1: PREDICTIVE ACCURACY OF MACHINE LEARNING CLASSIFIERS WITHIN INDIVIDUAL LANGUAGES

Regarding RQ1, our findings demonstrate that when machine learning classifiers are trained and evaluated on flaky tests using the same programming language, they can attain high predictive accuracy. This high accuracy is consistent with previous research showing the stability of these classifiers in identifying intricate patterns, especially in models like Random Forest (RF), Naive Bayes (NB), and k-Nearest Neighbors (kNN) [28], [33]. According to Ahmad et al. [2], who also highlight the significance of particular coding patterns in influencing test flakiness, our results are consistent with their findings, indicating that the classifiers in our study successfully identify these patterns within each language.

With F1-scores above 0.90 in languages like Java, Python, and C++, the strong performance of RF and kNN indicates that these classifiers are especially well-suited for identifying flaky tests within specific languages. According to Breiman's work on RF's resilience in high-dimensional data spaces, RF's ensemble approach and kNN's local dependency capture seem to manage the data complexity of flaky test features well [28]. On the other hand, as Allamanis et al. pointed out in their survey on the naturalness of code [24], the poorer performance of Linear Discriminant Analysis (LDA) indicates its limitations in handling non-linear relationships, which are crucial in flaky test detection where test behaviors may be influenced by a variety of factors.

These results confirm earlier findings that models trained in a single language achieve higher accuracy, highlighting the usefulness of language-specific models in flaky test detection. From a practical standpoint, this suggests that developers might find it advantageous to train distinct classifiers for every language in order to optimize detection accuracy, especially in settings where a single language is

the primary language. These results emphasize how crucial model selection is for detecting flaky tests. Within individual languages, non-linear and probabilistic models performed well, indicating that complex, non-linear dependencies that are difficult for simpler models like LDA to capture may be present in flaky test behaviors.

Commonalities: Our findings are consistent with earlier research showing that non-linear models like RF and kNN fared especially well across languages. According to Allamanis et al. [24], machine learning models trained in a single language frequently achieve optimal accuracy. Both models successfully captured flaky test patterns within each language. In situations where codebases are language-specific, this observation bolsters the useful suggestion to employ single-language models for flaky test detection.

Improvements: One difference between our study and previous research, like Bell et al. [9], which discovered that RF performed better than NB in flaky test detection, was that NB performed better across all languages. Our feature selection and preprocessing techniques may have contributed to our improved NB performance by enhancing NB's capacity to detect probabilistic dependencies in flaky tests [30]. Our study's strong results for NB imply that, with the use of improved feature engineering techniques, this model might provide even more advantages in flaky test detection.

Outliers: Compared to other classifiers, LDA performed noticeably worse, with lower precision, recall, and F1 scores, making it an outlier. This finding contrasts with previous research, which found that LDA occasionally performed moderately well for simpler flaky test patterns [9]. The conclusion that linear models might find it difficult to capture these subtleties is supported by the poor performance of LDA in our study, which probably reflects the complexity of our dataset and the non-linear dependencies in flaky test patterns.

B. RQ2: CROSS-LANGUAGE PREDICTIVE PERFORMANCE OF CLASSIFIERS

The generalizability of machine learning classifiers trained on one programming language and evaluated on another was investigated in RQ2. The limitations of cross-language flaky test detection are highlighted by our results, which demonstrate a significant decline in accuracy and F1-scores when models are applied across different languages. This result is in line with Ahmad et al. [2], who pointed out that because every language has different structural and syntactic subtleties, factors affecting test flakiness differ by language.

Interestingly, compared to other language pairs, the classifiers that were tested on C++ and trained on Java performed better across languages. This implies that more successful generalization may result from structural similarities between languages, especially those that share object-oriented features. Similar findings were made by Rahman and Rigby [13], who discovered that if two languages share programming paradigms, test dependencies in one language may partially apply to another. However, models demonstrated notable performance drops when tested on statically-typed languages like Java or C++ after training on Python or JavaScript, which differ syntactically and in typing systems. The findings of Gyori et al. [11], who described the difficulty of identifying faulty tests brought on by dynamic language-specific behaviors, are supported by this.

The practical drawback of applying a single model across languages in multilingualism settings is highlighted by these cross-language findings. To improve detection performance when adapting to multiple languages, our results instead point to the use of language-specific models or the investigation of transfer learning techniques [24]. Because each language has different flaky test patterns that call for customized detection models, this insight is especially pertinent for development teams using continuous integration in multi-language projects.

Luo et al.'s findings are consistent with the observation that cross-language models performed best when trained on structurally similar languages like Java and C++. Reference [4], who proposed that partial generalization between languages is facilitated by shared paradigms. For classifiers like Logistic Regression (LR) and RF, which demonstrated comparatively stable performance when moving from Java to C++, this similarity supports the notion that object-oriented languages may offer transferable patterns in erratic test data.

In contrast to previous research, our models' performance declined more dramatically when tested on statically-typed languages (like Java or C++) and trained on dynamic languages (like Python or JavaScript). In contrast, Gyori et al. [11] found that models trained in JavaScript showed moderate generalization when applied to C++. According to our findings, dynamic languages might introduce distinct patterns or test behaviors that are difficult to translate to statically-typed languages, especially in domains where dynamic typing adds variability, such as test setup and teardown.

Outliers: Unexpected findings emerged from cross-language testing using models trained with JavaScript; Random Forest and kNN models demonstrated comparatively high recall but decreased precision in JavaScript-Go configurations. This imbalance implies that applying JavaScript-trained models to Go increases the likelihood of false positives, even though some flaky test patterns may generalize well. According to Pinto et al. [14], this might be a reflection of the two languages' intrinsic differences in how asynchronous tests are handled.

Random Forest (RF) and **Logistic Regression (LR)** are the most dependable classifiers for cross-language flaky test detection, according to the accuracy heatmap (Figure 6). They consistently achieve higher accuracy across a variety of language pairs, including structurally similar pairs like Java-C++ and Java-Go. While classifiers like **Linear Discriminant Analysis (LDA)** and **Naive Bayes (NB)** perform poorly, especially when handling languages with different typing systems, like Python and C++, this shows their adaptability to a variety of language structures. It appears that language structure affects transferability because classifiers trained on languages with similar paradigms (like Java and C++)

generally generalize better across tests. These results indicate that while LDA and NB might need fine-tuning or specialized training for successful cross-language application, RF and LR are perfect for multi-language environments because they retain accuracy even in heterogeneous language setups.

C. RQ3: COMPARATIVE PERFORMANCE WITH PREVIOUS STUDIES

Our findings were contrasted with those of earlier research by Pinto et al. [14] and Camara et al. [15] in RQ3. Our study performed better than the original and replication studies, with notable improvements in F1-score and recall for classifiers such as NB, SVM, and kNN. These enhancements are the result of our improved feature selection procedure, which included sophisticated preprocessing methods that probably captured more subtle flaky test patterns. Such preprocessing, as highlighted by Bell et al. [9], can improve flaky test detection by reducing noise and highlighting important characteristics linked to test failures.

Compared to the Camara et al. replication study, some models, like LDA, did slightly worse in our investigation. This disparity may have resulted from our study's larger dataset, which added variability that LDA found difficult to account for. Bell and Elbaum's earlier findings [9] that non-linear approaches produce better recall and precision in flaky test scenarios are supported by this limitation, which implies that linear classifiers may be less effective for flaky test detection when trained on diverse datasets.

Models like NB and kNN performed well, as in earlier research, which demonstrates their adaptability to the flaky test domain. Our results validate the efficacy of probabilistic and proximity-based models in identifying flaky tests, supporting earlier findings that these models are well-suited for this task due to their capacity to capture subtle dependencies [43]. The importance of feature engineering in flaky test detection is highlighted by the superior performance of kNN and NB in our study. Our study's methodology improved the performance of classifiers that are known to be sensitive to feature selection by capturing subtle dependencies. This result validates Bell et al. [9]'s focus on sophisticated feature selection methods, which they stated can enhance recall in flaky test detection.

Improvements: Our research revealed that kNN and NB obtained the highest precision and F1-scores, in contrast to Pinto et al., who claimed the highest precision with RF. This disparity might be explained by our more selective feature selection, which probably improved NB and kNN performance by identifying dependencies specific to flaky test cases [24]. Our results imply that these classifiers might gain more from feature selection than RF, which could account for their better study results.

Unexpected Findings: Our LDA results were inconsistent with those of Camara et al., who used LDA to moderate effect. Our research, on the other hand, found that LDA had trouble with recall and precision, suggesting that it might not generalize well to datasets with greater diversity. Given that LDA might be better suited for identifying flaky tests with simple dependencies, this discrepancy might be a reflection of dataset diversity.

D. RQ4: INSIGHTS FROM STATISTICAL ANALYSIS

Figure 7 displays the accuracy t-test results, which indicate notable variations among specific classifier pairs. Across all programming languages, Random Forest consistently performed better than LDA, with a highly significant p-value of $p = 0.00003$. Likewise, Naive Bayes demonstrated a significant statistical advantage over LDA ($p = 0.00012$). Additionally, compared to LDA ($p = 0.00007$) and Decision Tree ($p = 0.03726$), the kNN classifier showed noticeably higher accuracy, demonstrating its dependability in identifying faulty tests.

Decision Tree and Logistic Regression performed similarly to the majority of classifiers, but poorly in comparison to Random Forest and Naive Bayes. The close comparison of Logistic Regression and Perceptron ($p = 0.03793$) reveals that they behave similarly, but the best-performing classifiers were Random Forest, Naive Bayes, and kNN.

The trends in accuracy are further supported by the t-test results for F1-scores, which are shown in Figure 8. While Naive Bayes and kNN also performed better than LDA ($p = 0.00007$ and $p = 0.00004$, respectively), Random Forest showed a statistically significant advantage over LDA ($p = 0.00002$). This supports the conclusion that these classifiers are very good at preserving recall and precision in the detection of flaky tests.

Furthermore, the statistical comparison showed that SMO performed poorly compared to Random Forest, kNN, and Naive Bayes, especially in F1-scores, even though it was competitive with Perceptron and Decision Tree. A moderate difference in the predictive performance of kNN and Perceptron is indicated by the p-values for SMO comparisons ($p = 0.03226$ and 0.00181, respectively).

In flaky test detection, these results demonstrate the statistical robustness of Random Forest, Naive Bayes, and kNN, which makes them excellent choices for both language-specific and cross-language scenarios. These results are important because they help identify classifiers that are statistically superior across metrics in addition to being effective.

VI. THEORETICAL AND PRACTICAL IMPLICATIONS

These results offer a theoretical foundation for choosing particular classifiers for tasks involving flaky test detection. While linear models might be better suited for certain situations with little variability, ensemble approaches and proximity-based classifiers seem to be the most successful at managing the diversity and unpredictability present in flaky test data. The implications of our study for software testing practices are significant. The high performance of language-specific models suggests that single-language classifiers should be prioritized for environments where a

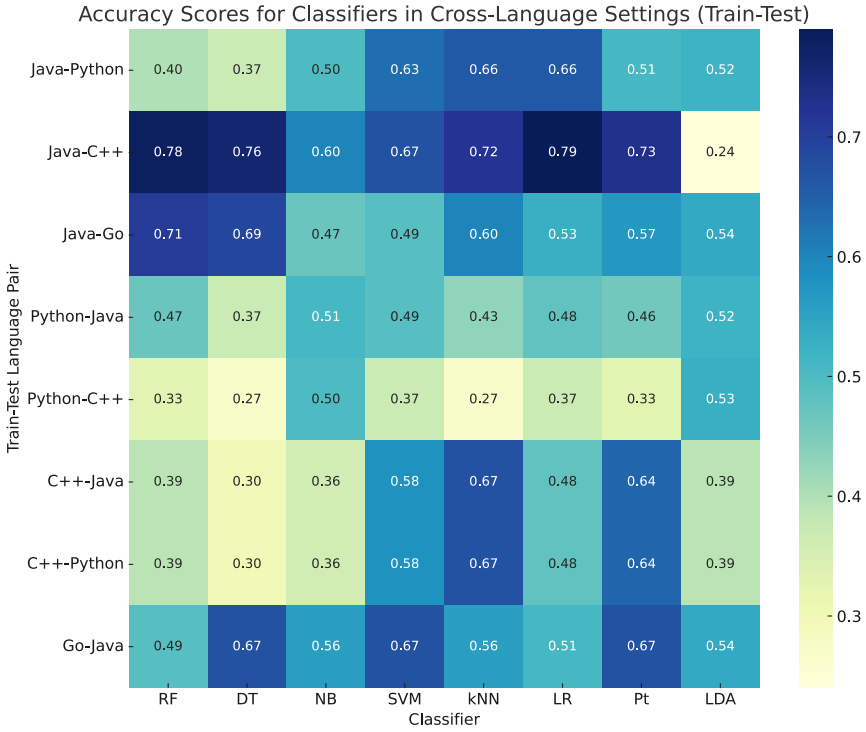


FIGURE 6. Cross language accuracy heat map.

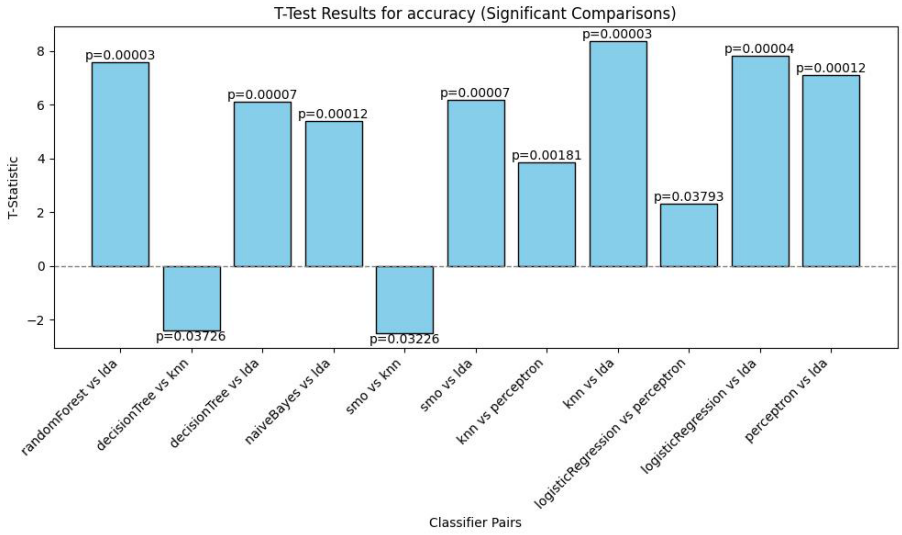


FIGURE 7. T-test result for accuracy.

specific language predominates, aligning with the recommendations by Ahmad et al. [2] regarding tailored flaky test solutions. Additionally, our cross-language findings imply that language-specific training is essential in multi-language settings to prevent loss of accuracy, as observed by Luo et al. [4]. For practical implementation, these insights suggest integrating language-specific models into CI pipelines to ensure reliable flaky test detection.

Our findings have important practical implications. First, Ahmad et al. [2] suggest that developers working in a single language should prioritize language-specific classifier training for flaky test detection due to the strong performance of language-specific models.

A. LIMITATIONS

According to Hilton et al. [6], our study may have overlooked external environmental factors that contribute to test flakiness because it concentrated primarily on structural features of code. The exclusion of transfer learning strategies is another drawback. Applying transfer learning could improve cross-language generalization for flaky test

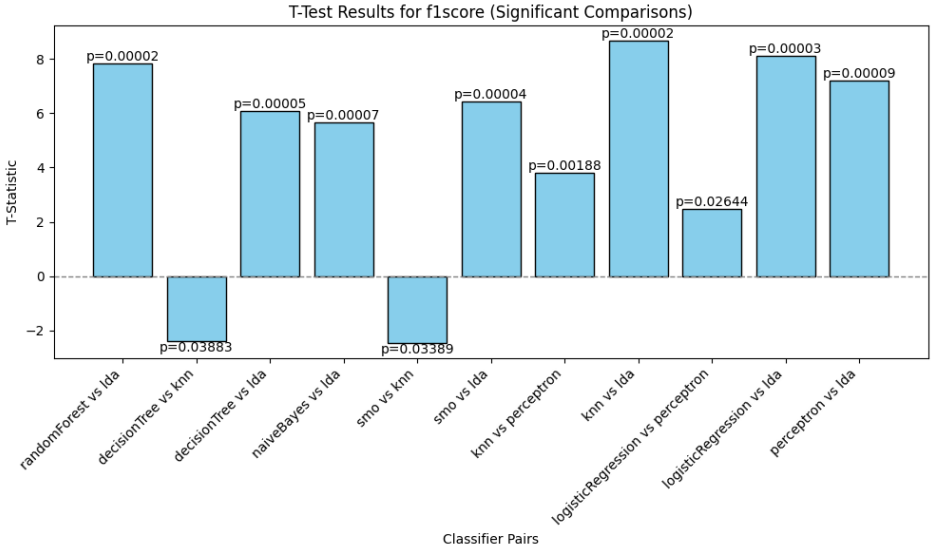


FIGURE 8. T-Test results for f1 Score.

detection, as it has demonstrated promise in related software testing domains [24].

It takes careful interpretation to reach an accuracy of 1.0 in our experiments, especially when it comes to Python, in order to guarantee the validity and dependability of the findings. This flawless accuracy could be explained by the Python dataset's uniqueness, which may include powerful patterns or features like particular keywords, coding structures, or styles that simplify the model's classification task. Although this shows how well our method can detect faulty tests, it is crucial to rule out potential problems like overfitting, data leakage, or an imbalanced dataset.

We carried out extra cross-validation to confirm the consistency of the results in order to guarantee that these findings are solid. Additionally, the high accuracy indicates that further research is necessary to verify the model's generalizability by testing its performance on datasets that have never been seen before or from different repositories. Although the accuracy of 1.0 is encouraging and demonstrates the strength of our methodology, we stress the need to interpret this result cautiously because it might represent dataset-specific features rather than generalizability. By taking these factors into account, we hope to improve the validity of our results and offer a more thorough assessment of the model's functionality.

B. FUTURE RESEARCH DIRECTIONS

Our results clearly indicate a number of areas which require further investigation. In line with Bell and Elbaum's recommendations, investigating transfer learning techniques may help close the cross-language performance gap found in this study [9]. Furthermore, our models' generalizability would be enhanced by enlarging the dataset to include additional programming languages and testing environments. In order to address non-code-related causes of flakiness, future research could also look at how external dependencies and runtime environments affect flaky test detection, as proposed by Hilton et al. [6].

Finally, incorporating ensemble techniques or deep learning models may improve flaky test detection even more, especially for languages with wildly fluctuating syntactic structures. This could expand on the dependency detection work of Gambi et al. [12] to create hybrid methods for identifying flaky tests.

VII. THREATS TO VALIDITY

This section discusses construct, internal, external, and conclusion validity as well as possible risks to the reliability of our results. By looking at these risks, we hope to offer an open evaluation of the variables that could affect the reliability and generalizability of our findings.

A. CONSTRUCT VALIDITY

Construct validity relates to the degree to which our study accurately captures the theoretical constructs under investigation, specifically the detection and generalizability of flaky tests across languages.

Our study's construct validity is primarily threatened by the feature selection and preprocessing techniques employed. Although previous research [9], [24] informed the selection of these techniques, they might not account for all variables that affect flaky tests, including environmental factors (e.g., memory usage, network latency) and other non-code-related factors. The significance of these external factors in flaky test causation is noted by Hilton et al. [6], who also suggest that a larger feature set could improve construct validity. To better capture the complex nature of flaky tests, future research could build on this by adding variables that represent environmental influences.

B. INTERNAL VALIDITY

Internal validity addresses the extent to which the study's results can be attributed to the independent variables rather than other confounding factors.

The differences in data structures between languages pose a risk to our study's internal validity because they could result in unanticipated biases. The performance of the model may be impacted by structural variations (such as object-oriented versus functional paradigms), even though we employed consistent classifiers and training-test splits across languages. The findings of Gyori et al. [11], who documented comparable difficulties in translating Java-trained models to other languages, are consistent with this variability. In order to better evaluate model adaptability, we selected language pairs that had both similarities and differences in order to lessen this threat.

C. EXTERNAL VALIDITY

External validity pertains to the generalizability of our findings beyond the specific context of this study.

The small number of programming languages included in our study poses a serious risk to external validity. Our results might not apply to other languages with different flaky test behaviors, even though we included a variety of languages (Java, Python, C++, Go, and JavaScript) that represent various paradigms. Languages like Swift or Rust might add special patterns that aren't shown here. According to Rahman and Rigby [13], who highlighted the importance of diverse language datasets in comprehending flaky test patterns, expanding the scope of future research could improve the generalizability of our findings.

Using open-source projects to create datasets is another risk to external validity. The generalizability of our findings may be impacted by flaky tests in proprietary software, which can result from various environmental or infrastructure-related factors (such as intricate continuous integration pipelines). Future research could address this by simulating industrial conditions or applying our models to datasets from proprietary software environments in order to more accurately evaluate model performance in various contexts.

D. CONCLUSION VALIDITY

Conclusion validity addresses the degree to which the conclusions drawn from our study are sound and supported by the data.

The selection of evaluation metrics, such as precision, recall, and F1-score, to gauge classifier performance poses a risk to the validity of the conclusions. Despite being common in classification tasks, these metrics might not adequately account for other pertinent elements of flaky test detection, like the effect of false positives and false negatives on developer workflows. Because it provides a balanced perspective of classifier performance, Boughorbel et al. [44] recommend using the Matthews Correlation Coefficient (MCC) as an alternative metric in imbalanced classification contexts. Future research could assess flaky test detection more thoroughly by incorporating MCC or comparable metrics.

Our comparison with earlier research by Pinto et al. [14] and Camara et al. [15] presents another possible risk. Even though we made an effort to duplicate important methodological components, minor variations in the preprocessing and dataset composition could affect how comparable our findings are. Our results should therefore be interpreted cautiously, and additional replication research is recommended to confirm these comparisons in other settings.

VIII. CONCLUSION

By methodically assessing the cross-language performance of multiple machine learning classifiers on various programming languages, this study advances the field of flaky test detection. Our results show that probabilistic methods like Logistic Regression and ensemble models like Random Forest consistently attain higher accuracy and generalizability across various language pairs. In cross-language contexts, where classifiers encounter difficulties because of structural and syntactic variances, this robustness is particularly important. We validate our approach and demonstrate improved performance, with notable gains in both precision and recall for flaky test detection, by comparing our results with those of Pinto et al. and Camara et al.

Important findings show that while linear classifiers such as Linear Discriminant Analysis have trouble with the non-linear complexities typical of flaky test behaviors across languages, classifiers trained on structurally similar languages show higher generalization accuracy. Our findings provide clear guidance on classifier selection for multi-language testing environments, despite the limitations of our study, which include potential overfitting within single-language training contexts and difficulties in cross-language adaptability due to different typing systems. To further enhance cross-language detection, future research should investigate adding more environmental features and transfer learning strategies.

ACKNOWLEDGMENT

The authors gratefully acknowledge the support provided by the Prince Sultan University, which covered the Article Processing Charges (APC) for this publication. The university's management commitment to advancing research and knowledge dissemination has been invaluable in successfully completing this research.

REFERENCES

[1] A. Ahmad, O. Leifler, and K. Sandahl, "An evaluation of machine learning methods for predicting flaky tests," in *Proc. 8th Int. Workshop Quant. Approaches Softw. Quality Conjunct. 27th Asia–Pacifc Softw. Eng. Conf. (APSEC)*, Dec. 2020, pp. 37–46.

[2] A. Ahmad, O. Leifler, and K. Sandahl, "Empirical analysis of factors and their effect on test flakiness—Practitioners' perceptions," 2019, *arXiv:1906.00673*.

[3] T. M. King, D. Santiago, J. Phillips, and P. J. Clarke, "Towards a Bayesian network model for predicting flaky automated tests," in *Proc. IEEE Int. Conf. Softw. Qual., Rel. Secur. Companion (QRS-C)*, Jul. 2018, pp. 100–107.

[4] Q. Luo, F. Hariri, L. Eloussi, and D. Marinov, "An empirical analysis of flaky tests," in *Proc. 22nd ACM SIGSOFT Int. Symp. Found. Softw. Eng.*, New York, NY, USA, Nov. 2014, pp. 643–653.

[5] A. Labuschagne, L. Inozemtseva, and R. Holmes, "Measuring the cost of regression testing in practice: A study of Java projects using continuous integration," in *Proc. 11th Joint Meeting Found. Softw. Eng.*, New York, NY, USA, Aug. 2017, pp. 821–830.

[6] M. Hilton, N. Nelson, T. Tunnell, D. Marinov, and D. Dig, "Trade-offs in continuous integration: Assurance, security, and flexibility," in *Proc. 11th Joint Meeting Found. Softw. Eng.*, New York, NY, USA, Aug. 2017, pp. 197–207.

[7] J. Micco. (2019). *Flaky Tests at Google and How we Mitigate Them*. Accessed: Apr. 15, 2019. [Online]. Available: https://testing.googleblog.com/2016/05/flaky-tests-at-google-and-how-we.html

[8] S. Thorve, C. Sreshtha, and N. Meng, "An empirical study of flaky tests in Android apps," in *Proc. IEEE Int. Conf. Softw. Maintenance Evol. (ICSME)*, Sep. 2018, pp. 534–538.

[9] J. Bell, O. Legunsen, M. Hilton, L. Eloussi, T. Yung, and D. Marinov, "DeFlaker: Automatically detecting flaky tests," in *Proc. IEEE/ACM 40th Int. Conf. Softw. Eng. (ICSE)*, May 2018, pp. 433–444.

[10] A. Shi, J. Bell, and D. Marinov, "Mitigating the effects of flaky tests on mutation testing," in *Proc. 28th ACM SIGSOFT Int. Symp. Softw. Test. Anal.*, New York, NY, USA, Jul. 2019, pp. 112–122.

[11] A. Gyori, B. Lambeth, A. Shi, O. Legunsen, and D. Marinov, "NonDex: A tool for detecting and debugging wrong assumptions on Java API specifications," in *Proc. 24th ACM SIGSOFT Int. Symp. Found. Softw. Eng.*, New York, NY, USA, Nov. 2016, pp. 993–997.

[12] A. Gambi, J. Bell, and A. Zeller, "Practical test dependency detection," in *Proc. IEEE 11th Int. Conf. Softw. Test., Verification Validation (ICST)*, Apr. 2018, pp. 1–11.

[13] M. T. Rahman and P. C. Rigby, "The impact of failing, flaky, and high failure tests on the number of crash reports associated with Firefox builds," in *Proc. 26th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, New York, NY, USA, Oct. 2018, pp. 857–862.

[14] G. Pinto, B. Miranda, S. Dissanayake, M. D'Amorim, C. Treude, and A. Bertolino, "What is the vocabulary of flaky tests?" in *Proc. IEEE/ACM 17th Int. Conf. Mining Softw. Repositories (MSR)*, New York, NY, USA, May 2020, pp. 492–502.

[15] B. H. P. Camara, M. A. G. Silva, A. T. Endo, and S. R. Vergilio, "What is the vocabulary of flaky tests? An extended replication," in *Proc. IEEE/ACM 29th Int. Conf. Program Comprehension (ICPC)*, May 2021, pp. 444–454.

[16] D. Marinov, J. Bell, and A. Shi, "Test dependency detection for practical flaky test mitigation," in *Proc. IEEE Int. Conf. Softw. Test., Verification Validation*, May 2019, pp. 1–11.

[17] X. Xu, Y. Dong, and F. Zhu, "A LiDAR assisted control module with high precision in parking scenarios for autonomous driving vehicle," 2021, *arXiv:2105.00398*.

[18] B. Miranda, G. Pinto, and S. Vergilio, "Feature engineering for flaky test detection: Lessons from the field," *Empirical Softw. Eng.*, vol. 26, pp. 543–567, Apr. 2021.

[19] S. Liviu. *A Machine Learning Solution for Detecting and Mitigating Flaky Tests—Medium.com*. Accessed: Mar. 19, 2025. [Online]. Available: https://medium.com/fitbit-tech-blog/a-machine-learning-solution-for-detecting-and-mitigating-flaky-tests-c5626ca7e853

[20] T. Wendler, A. Kumar, and A. Ahmad, "Cross-project flaky test detection: Challenges and solutions," *J. Softw. Test., Verification, Rel.*, vol. 35, pp. 121–145, May 2024.

[21] M. Cordy, I. Pashchenko, D. Kavaler, and M. Nagappan, "Studying flaky tests across projects: Replication challenges and opportunities," in *Proc. Int. Conf. Softw. Eng.*, 2021, pp. 123–133.

[22] Y. Lin, R. Wang, and M. Song, "Dynamic features for flaky test detection," in *Proc. Int. Conf. Softw. Maintenance Evol.*, 2024, pp. 145–153.

[23] R. K. Yin, *Case Study Research: Design and Methods*, 4th ed., Thousand Oaks, CA, USA: Calif Sage Publications, 2009.

[24] M. Allamanis, E. T. Barr, P. Devanbu, and C. Sutton, "A survey of machine learning for big code and naturalness," *ACM Comput. Surv.*, vol. 51, no. 4, pp. 1–37, Jul. 2019.

[25] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, U.K.: Cambridge Univ. Press, 2008.

[26] A. Ahmad, D. Rentas, D. Hasselqvist, P. Sandberg, K. Sandahl, and A. Vulgarakis, "Test case selection in continuous regression testing using machine learning: An industrial case study," in *Proc. IEEE 48th Annu. Comput., Softw., Appl. Conf. (COMPSAC)*, Jul. 2024, pp. 33–38.

[27] T. Joachims, "Text categorization with support vector machines: Learning with many relevant features," in *Proc. 10th Eur. Conf. Mach. Learn. (ECML)*. Cham, Switzerland: Springer, Jan. 1998, pp. 137–142.

[28] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.

[29] J. R. Quinlan, "Induction of decision trees," *Mach. Learn.*, vol. 1, no. 1, pp. 81–106, Mar. 1986.

[30] H. Zhang, "The optimality of naive Bayes," in *Proc. 17th Int. Florida Artif. Intell. Res. Soc. Conf.*, 2004, pp. 562–567.

[31] D. R. Cox, "The regression analysis of binary sequences," *J. Roy. Stat. Soc. B, Stat. Methodol.*, vol. 20, no. 2, pp. 215–232, Jul. 1958.

[32] C. Cortes and V. Vapnik, "Support-vector networks," *Mach. Learn.*, vol. 20, no. 3, pp. 273–297, Sep. 1995.

[33] T. M. Cover and P. E. Hart, "Nearest neighbor pattern classification," *IEEE Trans. Inf. Theory*, vol. IT-13, no. 1, pp. 21–27, Jan. 1967.

[34] F. Rosenblatt, "The perceptron: A probabilistic model for information storage and organization in the brain," *Psychol. Rev.*, vol. 65, no. 6, pp. 386–408, 1958.

[35] R. A. Fisher, "The use of multiple measurements in taxonomic problems," *Ann. Eugenics*, vol. 7, no. 2, pp. 179–188, Sep. 1936.

[36] T. Saito and M. Rehmsmeier, "Precision-recall curves and roc curves for machine learning evaluation," *PLoS ONE*, vol. 10, no. 3, 2015, Art. no. 0118432.

[37] J. Bell and S. Elbaum, "Improving recall of flaky test detection: A classification approach," in *Proc. Int. Symp. Softw. Test. Anal.*, 2018, pp. 193–203.

[38] J. Opitz and S. Burst, "F1-score in imbalanced classification problems: A comprehensive review," *Pattern Recognit.*, vol. 101, Jul. 2020, Art. no. 107328.

[39] C. M. Bishop, *Pattern Recognition and Machine Learning*. Cham, Switzerland: Springer, 2006.

[40] T. Fawcett, "An introduction to ROC analysis," *Pattern Recognit. Lett.*, vol. 27, no. 8, pp. 861–874, Jun. 2006.

[41] S. Boughorbel, F. Jarray, and M. El-Anbari, "The Matthews correlation coefficient (MCC) as an alternative metric in classification performance analysis," *BMC Bioinf.*, vol. 18, no. 1, pp. 1–13, 2017.

[42] P. Runeson, M. Host, A. Rainer, and B. Regnell, *Case Study Research in Software Engineering: Guidelines and Examples*, 1st ed., Hoboken, NJ, USA: Wiley, 2012.

[43] D. Zhang, J. Wang, and X. Zhao, "Estimating the uncertainty of average F1 scores," in *Proc. Int. Conf. Theory Inf. Retr.*, Northampton, MA, USA, Sep. 2015, pp. 317–320.

[44] D. Bowes, T. Hall, J. Petric, T. Shippey, and B. Turhan, "How good are my tests?" in *Proc. IEEE/ACM 8th Workshop Emerg. Trends Softw. Metrics (WETSoM)*, May 2017, pp. 9–14.

AZEEM AHMAD is a professional driving innovation in software engineering and AI/ML. With leadership roles at Ericsson, Sweden, and Linköping University, Sweden, he bridges academia and industry, collaborating globally and securing substantial research grants, supervising thesis, interns, and Ph.D. students. Leveraging his academic expertise, he brings a wealth of experience in teaching, mentoring, and open-source contributions, which have been recognized with awards and accolades. His dedication to fostering interdisciplinary collaboration and pushing the boundaries of technological advancement underscores his commitment to driving positive change.

XIN SUN received the M.Sc. degree in computer science from KTH, Sweden. He is currently pursuing the Ph.D. degree with Linköping University. His research interest includes quality assurance of AI components in software engineering.

MUHAMMAD RASHID NAEEM received the bachelor's degree in software engineering from International Islamic University, Pakistan, in 2012, the master's degree in software engineering from Chongqing University, China, in 2015, and the Ph.D. degree in software engineering from Sichuan University, China, in 2020. Currently, he is serving as an Assistant Professor with the College of Computer and Information Sciences, Prince Sultan University, Saudi Arabia. He is the author of various research articles published in reputed journals, such as Elsevier, Springer, Wiley, and IEEE publishers. His current research interests include software engineering and data science.

YASIR JAVED (Member, IEEE) received the Ph.D. degree from UNIMAS, Sarawak. He is currently a distinguished data scientist and a senior programmer/developer, boasting over 18 years of comprehensive experience spanning research, security programming, software development, project management, and analytics. His professional journey includes roles as an Analyst Programmer at prestigious centers within Prince Sultan University, where he contributes significantly to the ACM Professional Chapter in Saudi Arabia and the RIOTU Group, underscoring his commitment to advancing research and development in his field. He actively reviews for several journals and has managed numerous national and international research projects. His research interests include multifaceted, focusing on human-centric AI, ethical and privacy considerations, data analytics, forensics, smart cities, network security, education sustainability, instructional development, robotics, autonomous aerial vehicles, vehicular platoons, secure software development, signal processing, the IoT analytics, intelligent applications, and predictive computing inspired by AI. He received an Outstanding Ph.D. Student Award from UNIMAS, Sarawak, for his Ph.D. degree. In addition, he was awarded a Rector's Medal for his M.S. degree and a Distinguished Teaching Award. Recognized as a top researcher at PSU for his significant contributions, he has published over 100 peer-reviewed articles in prestigious journals, conference proceedings, and book chapters.

MOHAMMAD AKOUR received the bachelor's and master's degrees (Hons.) in computer information systems from Yarmouk University, in 2006 and 2008, respectively, and the Ph.D. degree (Hons.) in software engineering from NDSU, in 2013. In August 2008, he joined Yarmouk University as a Lecturer after graduating with his master's degree in computer information systems. In April 2013, he joined Yarmouk University again after graduating with his Ph.D. degree and was hired as the Vice Dean of Student Affairs, in 2018. In 2019, he got a sabbatical leave and joined Al Yamamah University, Riyadh, Saudi Arabia, as an Associate Professor in software engineering. He served as a Software Engineering Program Coordinator. In 2021, he joined Prince Sultan University as a Professor in software engineering. He served as the Computer Science Department Chair, from 2023 to 2024. He is currently a Professor in software engineering and the Software Engineering Department Chair. He is a member of the International Association of Engineers (IAENG). He served as the Head of Accreditation and Quality Assurance for two years and then was hired as the Director of the Computer and Information Center, in 2017. He serves as a keynote speaker, organizer, co-chair, and publicity chair for several IEEE conferences, and as ERB for more than ten ISI-indexed prestigious journals.

KRISTIAN SANDAHL (Senior Member, IEEE) is currently an Academic in the field of software engineering at Linköping University, Sweden. His work often involves close collaboration with industry partners, ensuring that his research has practical applications and real-world impact. He has contributed to the field through publications including studies on test case selection in continuous regression testing and the evaluation of test prioritization approaches. His recent projects include advancing methods for quality assurance of software generated by AI models, supported by substantial funding from Vinnova. In addition to his research, he is passionate about teaching and mentoring students, integrating project-based learning to provide hands-on experience in software engineering. His research interests include developing, operating, and maintaining large-scale software systems with an emphasis on quality and cost-effectiveness.